Regular versus Resource-Shared Design of an 8-bit RISC-V ALU

Ten Operations from One Adder and One Shifter:
Implementation and Hardware Verification on the Shrike-Lite FPGA

Samaira Khan
Entry No.: 2024EE10201

ELL305 Project Report
Supervisor: Prof. Subrat Kar

Department of Electrical Engineering
Indian Institute of Technology Delhi

13 November 2026

Copyright Assignment

Copyright © 2026 Indian Institute of Technology Delhi. All rights reserved.

I, Samaira Khan (Entry No. 2024EE10201), hereby assign all intellectual property rights (IPR) in this report and in the work it describes, including its text, tables, diagrams, source code, hardware designs and bitstreams, to the Indian Institute of Technology Delhi.

In the case of cited material, the IPR belongs to the respective authors of the sources, as acknowledged in the References.

Samaira Khan
Entry No. 2024EE10201
Date: 13 November 2026

Acknowledgements

I am thankful to Prof. Subrat Kar and Prof. Sumantra Dutta Roy, for their guidance. I am also grateful to Ms. Garima Singhal and to the laboratory staff, Mr. Utkarsh Roy and Mr. Deepak Priyadarshi for their help in the laboratory, including advice on the slide-switch input and on the use of the protocol analyzer.

I gratefully acknowledge the Department of Electrical Engineering and the Indian Institute of Technology Delhi for the use of their resources: the laboratory, the Shrike-Lite FPGA boards, the Digilent Analog Discovery 3 analyzer and the computing facilities.

Abstract

This report presents an 8-bit arithmetic logic unit (ALU) implementing the ten integer operations of the RISC-V RV32I base instruction set (ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT and SLTU), designed in Verilog and implemented on the Vicharak Shrike-Lite board, which combines a Renesas ForgeFPGA (SLG47910V) with an RP2040 microcontroller. Because the FPGA exposes only 14 user pins, the operands and opcode are delivered by the RP2040 as a 24-bit SPI-style serial frame, and the 8-bit result and status flags are observed on a protocol analyzer (Digilent Analog Discovery 3 with WaveForms). Two implementations were built: a *regular* ALU with one module per operation, and a *resource-shared* ALU that realises all ten operations with a single adder and a single shifter. Both designs were verified on hardware against a reference model and produced identical results for every test case, while the shared design uses 26 fewer LUTs (16.8% of the complete design).

Contents

1 Introduction **1**
1.1 Relevance to ELL305 1
1.2 Significance of the Project 1
1.3 Objectives 1

2 Platform and Tools **2**

3 ALU Specification **2**
3.1 Operations and opcode encoding 2
3.2 Status flags 3

4 System Architecture **3**
4.1 The pin-count constraint 3
4.2 Serial input frame 3
4.3 Block diagram 3
4.4 FPGA modules 4
4.5 Pin assignment 4

5 Regular ALU **4**

6 Resource-Shared ALU **4**
6.1 One adder for seven operations 5
6.2 One shifter for three shifts 5

7 Input Interface and Firmware **5**
7.1 Slide-switch input 5
7.2 Firmware 6

8 Tests Performed **6**
8.1 Test cases 6
8.2 Analyzer setup 7
8.3 Results of the eleven test cases 7
8.4 Regular versus shared ALU on hardware 7

9 Resource Utilisation **7**

10 Conclusion **8**

11 Proposed Evaluation Scheme **8**

References **9**

Glossary **10**

A Plagiarism Report **12**

B Reading Indices **13**
B.1 Method 13
B.2 Results 13
B.3 Interpretation 13

C Worked Examples of the Shared ALU **15**

D Source Code **16**
D.1 Resource-shared ALU 16
D.2 Regular ALU 19
D.3 RP2040 firmware 23

E I/O Planner Assignment **25**

F Wiring **25**

G WaveForms Configuration **26**

Index **27**

List of Figures

1 System architecture. The analyzer also monitors the three input wires, so its SPI decoder shows each frame alongside the result. 3

List of Tables

1 Hardware and software used. 2
2 Supported operations. 2
3 24-bit input frame (sent MSB first). 3
4 Final pin assignment. 4
5 How each operation uses the two shared units (C = carry flag, V = overflow flag). 5
6 Slide-switch connections (switch ON = 1). 6
7 Test cases and expected results. 7
8 Resource comparison from the Go Configure Resources Report (full design). 8
9 Proposed distribution of the demonstration marks. 8
10 Turnitin plagiarism check. 12
11 Reading indices of the report's running text (textstat 0.7.13). 13
12 Complete I/O Planner assignment. 25
13 Board jumpers. 25
14 Analog Discovery 3 connections. 25
15 PMOD header layout (column 1 at the 3.3 V end). 26
16 Logic analyzer settings. 26

1 Introduction

The ALU is the computational core of every processor [3, 4]. This project implements an 8-bit ALU whose operation set and opcode encoding follow the RISC-V RV32I integer instructions [1].

1.1 Relevance to ELL305

With the Table of Contents of the course textbook as reference, this project draws upon **Chapters 1 to 12 (Instructor: Prof. Sumantra Dutta Roy)** of the textbook **Basic Computer Architecture (Version 3.09)** by Prof. Smruti R. Sarangi [2] as its basis.

1.2 Significance of the Project

This project illustrates how the arithmetic logic unit, the computational core of a processor, is built from basic digital building blocks, and how its design choices affect the cost of the hardware. It draws together several concepts from the course textbook. First, binary number representation: unsigned and two's complement signed numbers, and the difference between carry, an unsigned condition, and overflow, a signed one, which the status flags make visible on the analyzer. Second, combinational arithmetic and logic circuits: the ripple-carry adder with its propagate and generate signals, subtraction by inverting an operand and adding one, magnitude comparison, and logical and arithmetic shifters. Third, instruction set architecture: the ten operations and their 4-bit opcode are taken directly from the RISC-V RV32I base instructions, so the ALU corresponds to the execute stage of a real processor. Fourth, sequential logic and timing: the serial loader uses a shift register, edge detection and two-stage synchronisers on an internal clock, and the outputs are registered so that all lines change on the same clock edge. Fifth, hardware description and implementation: the design is written in Verilog, synthesised, placed and routed for a real FPGA, and its resource use is read from the tool's report.

The project then extends this material. Textbooks usually present an ALU as a set of separate function units whose outputs are selected by a multiplexer; this structure was built as the regular design. The resource-shared design instead asks which hardware the operations have in common. It shows that the propagate and generate signals that every adder already computes are exactly the XOR and AND results, that OR follows from them, that subtraction and both comparisons reuse the same adder, and that a left shift is simply a mirrored right shift. All ten operations therefore come from one adder and one shifter. Measured on the FPGA, this saves 26 LUTs, or 16.8% of the complete design, while giving identical results on hardware. A textbook idea about sharing hardware thus becomes a measured engineering result.

Finally, the project illustrates practical constraints that a textbook rarely covers: a board with only 14 usable FPGA pins, which was solved with a serial input interface, and verification on real hardware with a protocol analyzer against an independent reference model.

1.3 Objectives

The objectives were:

1. implement all ten RV32I register–register ALU operations at 8-bit width;

2. fit the design onto the Shrike-Lite board;

3. verify the design on hardware with a protocol analyzer;

4. go beyond a textbook implementation by building a resource-shared ALU and quantifying its benefit.

2 Platform and Tools

Table 1: Hardware and software used.

Item	Details
FPGA board	Vicharak Shrike-Lite [7]: Renesas ForgeFPGA SLG47910V (Rev BB) [5], 1,120 five-input LUTs, 1,120 flip-flops, internal 50 MHz oscillator, plus an RP2040 microcontroller [8].
FPGA toolchain	Renesas Go Configure Software Hub (ForgeFPGA Workshop) [6]: synthesis, I/O planning, bitstream generation.
Microcontroller	RP2040 running Vicharak's MicroPython build [9], programmed from the Thonny IDE [10]; loads the bitstream with `shrike.flash()`.
Logic analyzer	Digilent Analog Discovery 3 with WaveForms [11] (Logic tool, SPI decoder).

3 ALU Specification

3.1 Operations and opcode encoding

The 4-bit opcode is formed exactly as a RISC-V decoder would form it, from bit 5 of the `funct7` field and the 3-bit `funct3` field [1]: op = {`funct7`[5], `funct3`}. Shift instructions use the low three bits of operand B as the shift amount, the 8-bit analogue of RV32 using `rs2[4:0]` for 32-bit data. The six unused opcodes produce a result of zero.

Table 2: Supported operations.

Operation	Opcode	Function	Notes
ADD	0000	$A + B$	carry and signed overflow flags
SUB	1000	$A - B$	carry flag = borrow; overflow flag
AND	0111	$A \wedge B$	bitwise
OR	0110	$A \vee B$	bitwise
XOR	0100	$A \oplus B$	bitwise
SLL	0001	$A \ll B$[2:0]	logical left shift
SRL	0101	$A \gg B$[2:0]	logical right shift (zero fill)
SRA	1101	$A \ggg B$[2:0]	arithmetic right shift (sign fill)
SLT	0010	1 if $A < B$ (signed)	result 00 or 01
SLTU	0011	1 if $A < B$ (unsigned)	result 00 or 01

3.2 Status flags

RISC-V itself has no flags register; the flags were added for observability:

- **Zero**: result is 00 (all operations).
- **Carry**: carry out of bit 7 for ADD; borrow ($A < B$ unsigned) for SUB; 0 otherwise.
- **Overflow**: signed overflow for ADD and SUB; 0 otherwise.

4 System Architecture

4.1 The pin-count constraint

A fully parallel ALU needs $8 + 8 + 4 = 20$ input pins and $8 + 3 = 11$ output pins, 31 in total, but the Shrike-Lite exposes only 14 FPGA pins on its headers. The design therefore receives its inputs serially: the RP2040 sends the opcode and both operands as one 24-bit frame over three wires, leaving the FPGA pins free for the result and flags.

4.2 Serial input frame

The frame uses SPI mode 0 (clock idles low, data sampled on the rising edge), MSB first, with an active-low chip select. A rising edge on `CS_N` marks the end of the frame and loads the operands.

Table 3: 24-bit input frame (sent MSB first).

Bits 23–20	Bits 19–16	Bits 15–8	Bits 7–0
0000	opcode	operand A	operand B

4.3 Block diagram

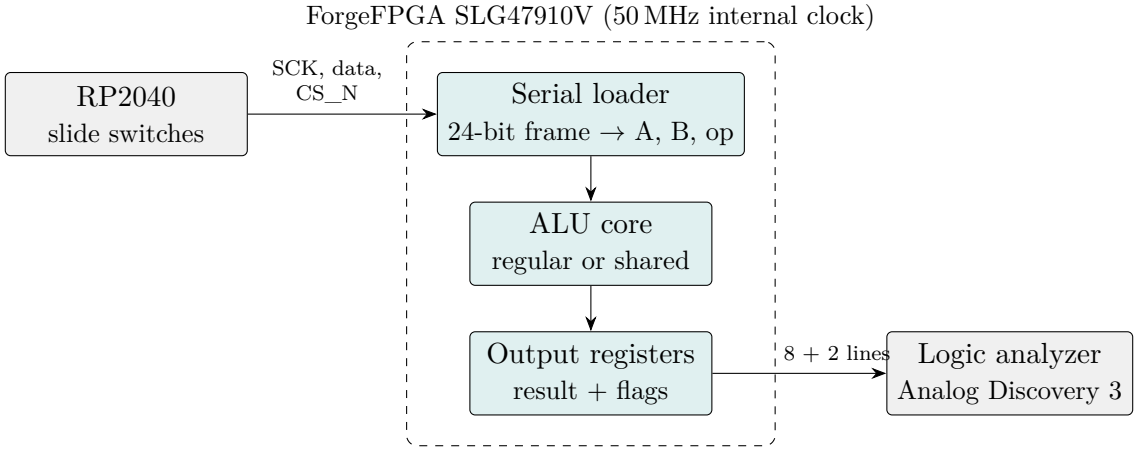


Figure 1: System architecture. The analyzer also monitors the three input wires, so its SPI decoder shows each frame alongside the result.

4.4 FPGA modules

Serial loader. Samples SCK, data and CS_N on the internal 50 MHz clock through two-stage synchronisers, shifts one bit per detected rising SCK edge while CS_N is low, and loads A, B and the opcode on the rising edge of CS_N. SCK is never used as a clock, which avoids clock-domain problems on the FPGA.

ALU core. Purely combinational; two interchangeable versions are described in Sections 5 and 6.

Output registers. Register the result and flags on one clock edge so that all output lines change simultaneously, giving clean transitions on the analyzer.

ForgeFPGA output pads require an explicit OE signal, so every output port has a companion `_oe` port driven to 1, and the internal oscillator is enabled through a `clk_en` port.

4.5 Pin assignment

Table 4 shows the final assignment; the complete I/O Planner settings are listed in Appendix E and the wiring in Appendix F.

Table 4: Final pin assignment.

Signal	FPGA pin	Connected to	Analyzer
`spi_sck` (in)	FPGA_IO18	RP2040 GPIO14 (jumper)	DIO 11
`spi_sdi` (in)	FPGA_IO7	RP2040 GPIO17 (jumper)	DIO 12
`spi_cs_n` (in)	FPGA_IO0	RP2040 GPIO16 (jumper)	DIO 13
`result[7:0]`	FPGA_IO8–IO15	PMOD header	DIO 0–7
`zero`	FPGA_IO1	header	DIO 8
`carry`	FPGA_IO2	header	DIO 9
`overflow`	FPGA_IO16	on-board LED	–
`clk` / `clk_en`	OSC_CLK / OSC_EN	internal oscillator	–

5 Regular ALU

The regular ALU follows the textbook structure: each operation is a separate module (`alu_add`, `alu_sub`, `alu_and`, `alu_or`, `alu_xor`, `alu_sll`, `alu_srl`, `alu_sra`, `alu_slt`, `alu_sltu`), all ten compute in parallel, and a multiplexer driven by the opcode selects one result. This structure is easy to read and debug, because every operation maps to exactly one module, but it duplicates hardware that several operations could share. The source code is listed in Appendix D.

6 Resource-Shared ALU

The shared ALU asks which hardware the ten operations have in common. All ten are built from just two units. Table 5 summarises how the opcode steers them.

Table 5: How each operation uses the two shared units (C = carry flag, V = overflow flag).

Operation	**Unit**	**Control**	**Result taken from**	**Flags**
ADD	adder	sub = 0	sum s	C = c_8, V
SUB	adder	sub = 1	sum s	C = $\neg c_8$, V
SLT	adder	sub = 1	$s_7 \oplus V$	–
SLTU	adder	sub = 1	$\neg c_8$ (borrow)	–
AND	adder	sub = 0	generate g	–
OR	adder	sub = 0	$p \vee g$	–
XOR	adder	sub = 0	propagate p	–
SLL	shifter	left = 1	mirror, shift, mirror	–
SRL	shifter	arith = 0	shift right, fill 0	–
SRA	shifter	arith = 1	shift right, fill sign	–

6.1 One adder for seven operations

An adder computes, for each bit, a propagate and a generate signal, which drive the carry chain [4]:

$$b' = b \oplus \text{sub}, \qquad p_i = a_i \oplus b'_i, \qquad g_i = a_i \wedge b'_i, \tag{1}$$

$$c_0 = \text{sub}, \qquad c_{i+1} = g_i \vee (p_i \wedge c_i), \qquad s_i = p_i \oplus c_i. \tag{2}$$

These internal signals are normally hidden, but they are exactly the logic results:

- XOR = p and AND = g (with sub = 0), and OR = $p \vee g$;
- SUB uses the same adder with B inverted and carry-in 1 (two's complement);
- SLT is the sign of $A - B$ corrected for overflow: SLT = $s_7 \oplus V$, where $V = c_8 \oplus c_7$;
- SLTU is the borrow of $A - B$: SLTU = $\neg c_8$.

6.2 One shifter for three shifts

A single right shifter with a selectable fill bit handles SRL (fill 0) and SRA (fill with the sign bit). A left shift is a right shift seen in a mirror: SLL reverses the bit order of A, shifts right, and reverses the result back. Bit reversal is pure wiring and costs no logic.

Worked examples for every test case are given in Appendix C, and the complete source code in Appendix D.

7 Input Interface and Firmware

7.1 Slide-switch input

The operands and opcode are set on 20 slide switches connected to free RP2040 pins (Table 6). Each switch connects its pin to a shared ground bus when ON; the RP2040's internal pull-ups supply the OFF level, so no external resistors are needed.

Table 6: Slide-switch connections (switch ON = 1).

Switches	RP2040 pins	Meaning
A0–A6	GPIO5–GPIO11	operand A, bits 0–6
A7	GPIO15	operand A, bit 7
B0–B7	GPIO18–GPIO25	operand B
op0–op3	GPIO26–GPIO29	opcode
common	GND	ground bus

7.2 Firmware

A MicroPython script (`alu_test.py`) running on the RP2040 loads the bitstream into the FPGA with `shrike.flash()`, reads the switches, and sends a new 24-bit frame whenever their setting changes, after checking that the reading has been stable for 20 ms (which also absorbs contact bounce). For every frame it prints the value predicted by an independent Python reference model. The RP2040 does not read the FPGA's result back: verification consists of comparing this prediction with the result captured by the analyzer.

Listing 1: Frame transmission in `alu_test.py`.

```
def send(op, a, b):
    frame = ((op & 0xF) << 16) | ((a & 0xFF) << 8) | (b & 0xFF)
    CS_N(0); time.sleep_us(10)
    for i in range(23, -1, -1):          # MSB first, SPI mode 0
        SDI((frame >> i) & 1); time.sleep_us(5)
        SCK(1); time.sleep_us(5)
        SCK(0)
    time.sleep_us(10); CS_N(1)           # rising edge -> FPGA loads A, B, op
```

8 Tests Performed

8.1 Test cases

Table 7 lists the tests performed on hardware and the expected analyzer readings. The operands target edge cases: signed overflow, a borrow, the difference between logical and arithmetic shifts, and signed versus unsigned comparison.

Table 7: Test cases and expected results.

Operation	SPI bytes	Result	Zero	Carry	Overflow (LED)
ADD 7F, 01	00 7F 01	80	0	0	1
SUB 10, 20	08 10 20	F0	0	1	0
AND F0, 3C	07 F0 3C	30	0	0	0
OR F0, 0F	06 F0 0F	FF	0	0	0
XOR AA, FF	04 AA FF	55	0	0	0
SLL 81, 03	01 81 03	08	0	0	0
SRL 81, 03	05 81 03	10	0	0	0
SRA 81, 03	0D 81 03	F0	0	0	0
SLT FF, 01	02 FF 01	01	0	0	0
SLTU FF, 01	03 FF 01	00	1	0	0
SLL 0D, 02	01 0D 02	34	0	0	0

8.2 Analyzer setup

In WaveForms' Logic tool, DIO 0–7 were grouped as a hexadecimal bus, DIO 8 and DIO 9 shown as the zero and carry flags, and an SPI decoder configured on DIO 11 (clock), DIO 12 (data) and DIO 13 (select, active low), sampling on the rising edge, 8 bits, MSB first. The trigger was set to the falling edge of DIO 13 (start of frame). The full settings are listed in Appendix G.

8.3 Results of the eleven test cases

All eleven tests produced the expected results on the analyzer, with the overflow LED lighting only for ADD 7F, 01. Out of these eleven tests, the carry flag was high only for SUB 10, 20 and the zero flag only for SLTU FF, 01, as expected.

8.4 Regular versus shared ALU on hardware

Both bitstreams were loaded in turn with identical wiring and test sequence; the analyzer showed identical results for all eleven test cases.

9 Resource Utilisation

Both designs were synthesised and implemented in Go Configure with identical interface logic, pin assignment and clocking; only the ALU core differs. Table 8 lists the figures from the Resources Report.

Table 8: Resource comparison from the Go Configure Resources Report (full design).

Resource	**Regular**	**Shared**	**Change**
CLB LUT5s (of 1,120)	155 (13.84%)	129 (11.52%)	−26 (−16.8%)
CLBs (of 140)	30 (21.43%)	27 (19.29%)	−3 (−10%)
Flip-flops	60	60	unchanged
CLB FFs	46	46	unchanged
IOB FFs	14	14	unchanged
GPIOs (of 19)	14	14	unchanged
OSCs	1	1	unchanged

Sharing one adder and one shifter removes 26 LUTs, 16.8% of the complete design. Because the serial loader, output registers and pin logic are identical in both designs, the whole saving comes from the ALU core, so the reduction within the core itself is larger than 16.8%. The flip-flop count is unchanged, as expected: the ALU core is purely combinational, and the change affects only combinational logic.

10 Conclusion

An 8-bit ALU implementing all ten RV32I integer operations was designed, implemented on the Shrike-Lite board and verified on hardware. A serial input interface solved the board's pin limitation and allowed the protocol analyzer to display each operation's inputs alongside its result. The resource-shared ALU, built from a single adder and a single shifter, produced results identical to the regular design in every hardware test while using 26 fewer LUTs, 16.8% of the complete design, according to the Go Configure Resources Report. A slide-switch interface allows any operation and operands to be entered and verified on the analyzer.

11 Proposed Evaluation Scheme

The 18 marks for the demonstration are proposed to be distributed equally between two parts, as shown in Table 9.

Table 9: Proposed distribution of the demonstration marks.

Part	**Demonstration**	**Marks**
1	Any one of the ten operations, chosen by the student	9
2	One operation chosen by the examiner	9
	Total	**18**

In both parts, an operation counts as demonstrated when its operands and opcode are set on the slide switches and the result captured on the protocol analyzer matches the expected value, together with the corresponding status flags.

References

[1] A. Waterman and K. Asanović (eds.), *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Document Version 20191213, RISC-V Foundation, 2019.

[2] S. R. Sarangi, *Basic Computer Architecture*, Version 3.09. Freely available online; print edition: White Falcon Publishing, 2021. Available: `https://www.cse.iitd.ac.in/~srsarangi/archbooksoft.html`

[3] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, 2nd ed., Morgan Kaufmann, 2020.

[4] S. L. Harris and D. M. Harris, *Digital Design and Computer Architecture: RISC-V Edition*, Morgan Kaufmann, 2021.

[5] Renesas Electronics, *SLG47910 ForgeFPGA Datasheet*. Available: `https://www.renesas.com`

[6] Renesas Electronics, *Go Configure Software Hub*. Available: `https://www.renesas.com`

[7] Vicharak, *Shrike: documentation and examples*. Available: `https://github.com/vicharak-in/shrike`

[8] Raspberry Pi Ltd, *RP2040 Datasheet*. Available: `https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf`

[9] MicroPython project, *MicroPython documentation*. Available: `https://docs.micropython.org`

[10] Thonny, *Thonny: Python IDE for beginners*. Available: `https://thonny.org`

[11] Digilent, *Analog Discovery 3 and WaveForms reference manuals*. Available: `https://digilent.com/reference`

[12] Analyze My Writing, *What is a Readability Index?* Available: `https://www.analyzemywriting.com/readability_indices.html`

[13] textstat contributors, *textstat: Python package to calculate readability statistics of a text object*, version 0.7.13. Available: `https://github.com/textstat/textstat`

Glossary

ALU Arithmetic logic unit: the part of a processor that performs arithmetic and logical operations.

bitstream The configuration file that programs an FPGA.

CLB Configurable logic block: a group of LUTs and flip-flops in the FPGA fabric.

CS_N Active-low chip select: frames an SPI transfer; its rising edge marks the end of a frame.

FPGA Field-programmable gate array: a chip whose logic is configured after manufacture by loading a bitstream.

generate In an adder, the signal $g_i = a_i \wedge b_i$ indicating that bit i produces a carry by itself.

GPIO General-purpose input/output: a pin that can be configured as an input or an output.

IOB Input/output block: the circuitry connecting an FPGA pin to the internal logic.

ISA Instruction set architecture: the instructions and encodings a processor understands.

LSB Least significant bit.

LUT Look-up table: the basic logic element of an FPGA; a LUT5 implements any Boolean function of five inputs.

MSB Most significant bit.

OE Output enable: the signal that switches on a pin's output driver.

overflow A signed result that does not fit in the available bits, e.g. two positive numbers giving a negative sum.

PMOD A 12-pin peripheral header standard; on the Shrike-Lite it carries FPGA_IO8–IO15.

propagate In an adder, the signal $p_i = a_i \oplus b_i$ indicating that bit i passes an incoming carry on.

readability index A formula estimating the years of education needed to read a text easily.

RTL Register-transfer level: a hardware description in terms of registers and the logic between them.

RV32I The 32-bit base integer instruction set of RISC-V.

SCK Serial clock line of an SPI link.

SPI Serial Peripheral Interface: a synchronous serial protocol with clock, data and chip-select lines.

two's complement The standard representation of signed binary numbers, in which the negative of a number is formed by inverting all its bits and adding 1.

A Plagiarism Report

This report was checked for plagiarism on turnitin.com. The summary below matches the submission made there; the full Turnitin report follows.

Table 10: Turnitin plagiarism check.

Item	**Value**
Submission ID	trn:oid:::1:3710129477
Date of check	Nov 13, 2026, 1:16 PM GMT+5:30
Overall similarity	11%

SAMAIRA KHAN

Project_2024EE10201_Samaira_Khan.pdf

Project

ELL305 2601 Computer Architecture

India Institute of Technology Delhi

Document Details

Submission ID
trn:oid:::1:3710129477

Submission Date
Nov 13, 2026, 1:15 PM GMT+5:30

Download Date
Nov 13, 2026, 1:17 PM GMT+5:30

File Name
Project_2024EE10201_Samaira_Khan.pdf

File Size
428.3 KB

34 Pages

8,053 Words

37,008 Characters

11% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text

Match Groups

62 Not Cited or Quoted 11%
Matches with neither in-text citation nor quotation marks

4 Missing Quotations 1%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

8% Internet sources
5% Publications
6% Submitted works (Student Papers)

Match Groups

- **62** Not Cited or Quoted 11%
  Matches with neither in-text citation nor quotation marks
- **4** Missing Quotations 1%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

8% Internet sources
5% Publications
6% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Student papers	
Yonsei University		1%
2	Internet	
www.renesas.com		1%
3	Internet	
tjrwjs147.tistory.com		<1%
4	Student papers	
University of Central Florida		<1%
5	Student papers	
University of Florida		<1%
6	Student papers	
University of Northumbria at Newcastle		<1%
7	Internet	
download.bibis.ir		<1%
8	Internet	
eecs.wsu.edu		<1%
9	Internet	
www.circuitstate.com		<1%
10	Internet	
ssy1938010014.github.io		<1%

11	Internet	
namberino.github.io		<1%
12	Student papers	
American Public University System		<1%
13	Student papers	
California State University, Sacramento		<1%
14	Internet	
huggingface.co		<1%
15	Student papers	
Oregon State University		<1%
16	Student papers	
Rochester Institute of Technology		<1%
17	Internet	
www.slideshare.net		<1%
18	Student papers	
Universidade de Brasília, UnB		<1%
19	Internet	
dranirbanchakraborty.com		<1%
20	Publication	
Daniel Page. "Practical Introduction to Computer Architecture", Springer Nature, ...		<1%
21	Publication	
Pong P. Chu. "Embedded SoPC Design with Nios II Processor and Verilog Example...		<1%
22	Publication	
Wasan Abdulwahab Mohammed, Husniza Husni. "Reading apps for children: Rea...		<1%
23	Internet	
fulltopic.github.io		<1%
24	Internet	
www.farnell.com		<1%

25	Internet	
www.ictessentials.com		<1%
26	Internet	
production.soa.org		<1%
27	Internet	
community.infineon.com		<1%
28	Internet	
mafiadoc.com		<1%
29	Internet	
yspm.edu.in		<1%
30	Internet	
www.wattpad.com		<1%
31	Internet	
www.coursehero.com		<1%
32	Internet	
www.ic-on-line.cn		<1%
33	Publication	
Hafiz Md. Hasan Babu. "Multiple-Valued Computing in Quantum Molecular Biolog...		<1%
34	Student papers	
Higher Education Commission Pakistan		<1%
35	Publication	
Parhami, Behrooz. "Computer Architecture", Oxford University Press		<1%
36	Internet	
chipverify.com		<1%
37	Internet	
indico.ictp.it		<1%
38	Internet	
www.ajopac.com		<1%

Regular versus Resource-Shared Design of an 8-bit RISC-V ALU

Ten Operations from One Adder and One Shifter:
Implementation and Hardware Verification on the Shrike-Lite FPGA

Samaira Khan
Entry No.: 2024EE10201

ELL305 Project Report
Supervisor: Prof. Subrat Kar

Department of Electrical Engineering
Indian Institute of Technology Delhi

13 November 2026

Copyright Assignment

Copyright © 2026 Indian Institute of Technology Delhi. All rights reserved.

I, Samaira Khan (Entry No. 2024EE10201), hereby assign all intellectual property rights (IPR) in this report and in the work it describes, including its text, tables, diagrams, source code, hardware designs and bitstreams, to the Indian Institute of Technology Delhi.

In the case of cited material, the IPR belongs to the respective authors of the sources, as acknowledged in the References.

Samaira Khan
Entry No. 2024EE10201
Date: 13 November 2026

Acknowledgements

I am thankful to Prof. Subrat Kar and Prof. Sumantra Dutta Roy, for their guidance. I am also grateful to Ms. Garima Singhal and to the laboratory staff, Mr. Utkarsh Roy and Mr. Deepak Priyadarshi for their help in the laboratory, including advice on the slide-switch input and on the use of the protocol analyzer.

28 I gratefully acknowledge the Department of Electrical Engineering and the Indian Institute of Technology Delhi for the use of their resources: the laboratory, the Shrike-Lite FPGA boards, the Digilent Analog Discovery 3 analyzer and the computing facilities.

Abstract

This report presents an 8-bit arithmetic logic unit (ALU) implementing the ten integer operations of the RISC-V RV32I base instruction set (ADD, SUB, AND, OR, XOR, SLL, SRL, SRA, SLT and SLTU), designed in Verilog and implemented on the Vicharak Shrike-Lite board, which combines a Renesas ForgeFPGA (SLG47910V) with an RP2040 microcontroller. Because the FPGA exposes only 14 user pins, the operands and opcode are delivered by the RP2040 as a 24-bit SPI-style serial frame, and the 8-bit result and status flags are observed on a protocol analyzer (Digilent Analog Discovery 3 with WaveForms). Two implementations were built: a *regular* ALU with one module per operation, and a *resource-shared* ALU that realises all ten operations with a single adder and a single shifter. Both designs were verified on hardware against a reference model and produced identical results for every test case, while the shared design uses 26 fewer LUTs (16.8% of the complete design).

Contents

1 Introduction **1**
1.1 Relevance to ELL305 1
1.2 Significance of the Project 1
1.3 Objectives 1

2 Platform and Tools **2**

3 ALU Specification **2**
3.1 Operations and opcode encoding 2
3.2 Status flags 3

4 System Architecture **3**
4.1 The pin-count constraint 3
4.2 Serial input frame 3
4.3 Block diagram 3
4.4 FPGA modules 4
4.5 Pin assignment 4

5 Regular ALU **4**

6 Resource-Shared ALU **4**
6.1 One adder for seven operations 5
6.2 One shifter for three shifts 5

7 Input Interface and Firmware **5**
7.1 Slide-switch input 5
7.2 Firmware 6

8 Tests Performed **6**
8.1 Test cases 6
8.2 Analyzer setup 7
8.3 Results of the eleven test cases 7
8.4 Regular versus shared ALU on hardware 7

9 Resource Utilisation **7**

10 Conclusion **8**

11 Proposed Evaluation Scheme **8**

References **9**

Glossary **10**

A Plagiarism Report **12**

B Reading Indices **13**
B.1 Method 13
B.2 Results 13
B.3 Interpretation 13

C Worked Examples of the Shared ALU **15**

D Source Code **16**
D.1 Resource-shared ALU 16
D.2 Regular ALU 19
D.3 RP2040 firmware 23

E I/O Planner Assignment **25**

F Wiring **25**

G WaveForms Configuration **26**

Index **27**

List of Figures

1 System architecture. The analyzer also monitors the three input wires, so its SPI decoder shows each frame alongside the result. 3

List of Tables

1 Hardware and software used. 2
2 Supported operations. 2
3 24-bit input frame (sent MSB first). 3
4 Final pin assignment. 4
5 How each operation uses the two shared units (C = carry flag, V = overflow flag). 5
6 Slide-switch connections (switch ON = 1). 6
7 Test cases and expected results. 7
8 Resource comparison from the Go Configure Resources Report (full design). 8
9 Proposed distribution of the demonstration marks. 8
10 Turnitin plagiarism check. 12
11 Reading indices of the report's running text (textstat 0.7.13). 13
12 Complete I/O Planner assignment. 25
13 Board jumpers. 25
14 Analog Discovery 3 connections. 25
15 PMOD header layout (column 1 at the 3.3 V end). 26
16 Logic analyzer settings. 26

1 Introduction

The ALU is the computational core of every processor [3, 4]. This project implements an 8-bit ALU whose operation set and opcode encoding follow the RISC-V RV32I integer instructions [1].

1.1 Relevance to ELL305

With the Table of Contents of the course textbook as reference, this project draws upon **Chapters 1 to 12 (Instructor: Prof. Sumantra Dutta Roy)** of the textbook **Basic Computer Architecture (Version 3.09)** by Prof. Smruti R. Sarangi [2] as its basis.

1.2 Significance of the Project

This project illustrates how the arithmetic logic unit, the computational core of a processor, is built from basic digital building blocks, and how its design choices affect the cost of the hardware. It draws together several concepts from the course textbook. First, binary number representation: unsigned and two's complement signed numbers, and the difference between carry, an unsigned condition, and overflow, a signed one, which the status flags make visible on the analyzer. Second, combinational arithmetic and logic circuits: the ripple-carry adder with its propagate and generate signals, subtraction by inverting an operand and adding one, magnitude comparison, and logical and arithmetic shifters. Third, instruction set architecture: the ten operations and their 4-bit opcode are taken directly from the RISC-V RV32I base instructions, so the ALU corresponds to the execute stage of a real processor. Fourth, sequential logic and timing: the serial loader uses a shift register, edge detection and two-stage synchronisers on an internal clock, and the outputs are registered so that all lines change on the same clock edge. Fifth, hardware description and implementation: the design is written in Verilog, synthesised, placed and routed for a real FPGA, and its resource use is read from the tool's report.

The project then extends this material. Textbooks usually present an ALU as a set of separate function units whose outputs are selected by a multiplexer; this structure was built as the regular design. The resource-shared design instead asks which hardware the operations have in common. It shows that the propagate and generate signals that every adder already computes are exactly the XOR and AND results, that OR follows from them, that subtraction and both comparisons reuse the same adder, and that a left shift is simply a mirrored right shift. All ten operations therefore come from one adder and one shifter. Measured on the FPGA, this saves 26 LUTs, or 16.8% of the complete design, while giving identical results on hardware. A textbook idea about sharing hardware thus becomes a measured engineering result.

Finally, the project illustrates practical constraints that a textbook rarely covers: a board with only 14 usable FPGA pins, which was solved with a serial input interface, and verification on real hardware with a protocol analyzer against an independent reference model.

1.3 Objectives

The objectives were:

1. implement all ten RV32I register–register ALU operations at 8-bit width;

2. fit the design onto the Shrike-Lite board;

3. verify the design on hardware with a protocol analyzer;

4. go beyond a textbook implementation by building a resource-shared ALU and quantifying its benefit.

2 Platform and Tools

Table 1: Hardware and software used.

Item	Details
FPGA board	Vicharak Shrike-Lite [7]: Renesas ForgeFPGA SLG47910V (Rev BB) [5], 1,120 five-input LUTs, 1,120 flip-flops, internal 50 MHz oscillator, plus an RP2040 microcontroller [8].
FPGA toolchain	Renesas Go Configure Software Hub (ForgeFPGA Workshop) [6]: synthesis, I/O planning, bitstream generation.
Microcontroller	RP2040 running Vicharak's MicroPython build [9], programmed from the Thonny IDE [10]; loads the bitstream with `shrike.flash()`.
Logic analyzer	Digilent Analog Discovery 3 with WaveForms [11] (Logic tool, SPI decoder).

3 ALU Specification

3.1 Operations and opcode encoding

The 4-bit opcode is formed exactly as a RISC-V decoder would form it, from bit 5 of the `funct7` field and the 3-bit `funct3` field [1]: op = {`funct7`[5], `funct3`}. Shift instructions use the low three bits of operand B as the shift amount, the 8-bit analogue of RV32 using `rs2[4:0]` for 32-bit data. The six unused opcodes produce a result of zero.

Table 2: Supported operations.

Operation	Opcode	Function	Notes
ADD	0000	$A + B$	carry and signed overflow flags
SUB	1000	$A - B$	carry flag = borrow; overflow flag
AND	0111	$A \wedge B$	bitwise
OR	0110	$A \vee B$	bitwise
XOR	0100	$A \oplus B$	bitwise
SLL	0001	$A \ll B[2{:}0]$	logical left shift
SRL	0101	$A \gg B[2{:}0]$	logical right shift (zero fill)
SRA	1101	$A \ggg B[2{:}0]$	arithmetic right shift (sign fill)
SLT	0010	1 if $A < B$ (signed)	result 00 or 01
SLTU	0011	1 if $A < B$ (unsigned)	result 00 or 01

3.2 Status flags

RISC-V itself has no flags register; the flags were added for observability:

- **Zero**: result is 00 (all operations).
- **Carry**: carry out of bit 7 for ADD; borrow ($A < B$ unsigned) for SUB; 0 otherwise.
- **Overflow**: signed overflow for ADD and SUB; 0 otherwise.

4 System Architecture

4.1 The pin-count constraint

A fully parallel ALU needs $8 + 8 + 4 = 20$ input pins and $8 + 3 = 11$ output pins, 31 in total, but the Shrike-Lite exposes only 14 FPGA pins on its headers. The design therefore receives its inputs serially: the RP2040 sends the opcode and both operands as one 24-bit frame over three wires, leaving the FPGA pins free for the result and flags.

4.2 Serial input frame

The frame uses SPI mode 0 (clock idles low, data sampled on the rising edge), MSB first, with an active-low chip select. A rising edge on `CS_N` marks the end of the frame and loads the operands.

Table 3: 24-bit input frame (sent MSB first).

Bits 23–20	Bits 19–16	Bits 15–8	Bits 7–0
0000	opcode	operand A	operand B

4.3 Block diagram

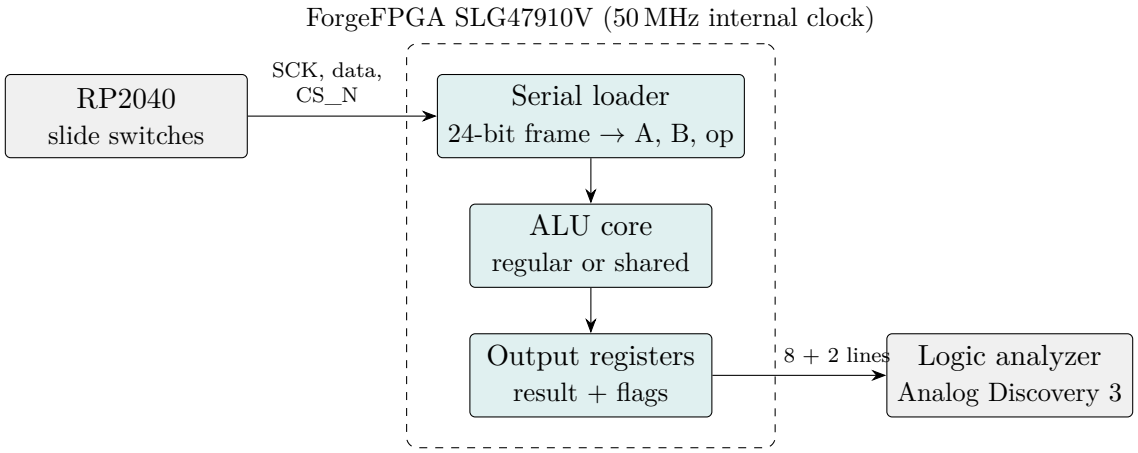


Figure 1: System architecture. The analyzer also monitors the three input wires, so its SPI decoder shows each frame alongside the result.

4.4 FPGA modules

Serial loader. Samples SCK, data and CS_N on the internal 50 MHz clock through two-stage synchronisers, shifts one bit per detected rising SCK edge while CS_N is low, and loads A, B and the opcode on the rising edge of CS_N. SCK is never used as a clock, which avoids clock-domain problems on the FPGA.

ALU core. Purely combinational; two interchangeable versions are described in Sections 5 and 6.

Output registers. Register the result and flags on one clock edge so that all output lines change simultaneously, giving clean transitions on the analyzer.

ForgeFPGA output pads require an explicit OE signal, so every output port has a companion `_oe` port driven to 1, and the internal oscillator is enabled through a `clk_en` port.

4.5 Pin assignment

Table 4 shows the final assignment; the complete I/O Planner settings are listed in Appendix E and the wiring in Appendix F.

Table 4: Final pin assignment.

Signal	FPGA pin	Connected to	Analyzer
`spi_sck` (in)	FPGA_IO18	RP2040 GPIO14 (jumper)	DIO 11
`spi_sdi` (in)	FPGA_IO7	RP2040 GPIO17 (jumper)	DIO 12
`spi_cs_n` (in)	FPGA_IO0	RP2040 GPIO16 (jumper)	DIO 13
`result[7:0]`	FPGA_IO8–IO15	PMOD header	DIO 0–7
`zero`	FPGA_IO1	header	DIO 8
`carry`	FPGA_IO2	header	DIO 9
`overflow`	FPGA_IO16	on-board LED	–
`clk` / `clk_en`	OSC_CLK / OSC_EN	internal oscillator	–

5 Regular ALU

The regular ALU follows the textbook structure: each operation is a separate module (`alu_add`, `alu_sub`, `alu_and`, `alu_or`, `alu_xor`, `alu_sll`, `alu_srl`, `alu_sra`, `alu_slt`, `alu_sltu`), all ten compute in parallel, and a multiplexer driven by the opcode selects one result. This structure is easy to read and debug, because every operation maps to exactly one module, but it duplicates hardware that several operations could share. The source code is listed in Appendix D.

6 Resource-Shared ALU

The shared ALU asks which hardware the ten operations have in common. All ten are built from just two units. Table 5 summarises how the opcode steers them.

Table 5: How each operation uses the two shared units (C = carry flag, V = overflow flag).

Operation	Unit	Control	Result taken from	Flags
ADD	adder	sub = 0	sum s	C = c_8, V
SUB	adder	sub = 1	sum s	C = $\neg c_8$, V
SLT	adder	sub = 1	$s_7 \oplus V$	–
SLTU	adder	sub = 1	$\neg c_8$ (borrow)	–
AND	adder	sub = 0	generate g	–
OR	adder	sub = 0	$p \vee g$	–
XOR	adder	sub = 0	propagate p	–
SLL	shifter	left = 1	mirror, shift, mirror	–
SRL	shifter	arith = 0	shift right, fill 0	–
SRA	shifter	arith = 1	shift right, fill sign	–

6.1 One adder for seven operations

An adder computes, for each bit, a propagate and a generate signal, which drive the carry chain [4]:

$$b' = b \oplus \text{sub}, \qquad p_i = a_i \oplus b'_i, \qquad g_i = a_i \wedge b'_i, \tag{1}$$

$$c_0 = \text{sub}, \qquad c_{i+1} = g_i \vee (p_i \wedge c_i), \qquad s_i = p_i \oplus c_i. \tag{2}$$

These internal signals are normally hidden, but they are exactly the logic results:

- XOR = p and AND = g (with sub = 0), and OR = $p \vee g$;
- SUB uses the same adder with B inverted and carry-in 1 (two's complement);
- SLT is the sign of $A - B$ corrected for overflow: SLT = $s_7 \oplus V$, where $V = c_8 \oplus c_7$;
- SLTU is the borrow of $A - B$: SLTU = $\neg c_8$.

6.2 One shifter for three shifts

A single right shifter with a selectable fill bit handles SRL (fill 0) and SRA (fill with the sign bit). A left shift is a right shift seen in a mirror: SLL reverses the bit order of A, shifts right, and reverses the result back. Bit reversal is pure wiring and costs no logic.

Worked examples for every test case are given in Appendix C, and the complete source code in Appendix D.

7 Input Interface and Firmware

7.1 Slide-switch input

The operands and opcode are set on 20 slide switches connected to free RP2040 pins (Table 6). Each switch connects its pin to a shared ground bus when ON; the RP2040's internal pull-ups supply the OFF level, so no external resistors are needed.

Table 6: Slide-switch connections (switch ON = 1).

Switches	RP2040 pins	Meaning
A0–A6	GPIO5–GPIO11	operand A, bits 0–6
A7	GPIO15	operand A, bit 7
B0–B7	GPIO18–GPIO25	operand B
op0–op3	GPIO26–GPIO29	opcode
common	GND	ground bus

7.2 Firmware

A MicroPython script (`alu_test.py`) running on the RP2040 loads the bitstream into the FPGA with `shrike.flash()`, reads the switches, and sends a new 24-bit frame whenever their setting changes, after checking that the reading has been stable for 20 ms (which also absorbs contact bounce). For every frame it prints the value predicted by an independent Python reference model. The RP2040 does not read the FPGA's result back: verification consists of comparing this prediction with the result captured by the analyzer.

Listing 1: Frame transmission in `alu_test.py`.

```
def send(op, a, b):
    frame = ((op & 0xF) << 16) | ((a & 0xFF) << 8) | (b & 0xFF)
    CS_N(0); time.sleep_us(10)
    for i in range(23, -1, -1):          # MSB first, SPI mode 0
        SDI((frame >> i) & 1); time.sleep_us(5)
        SCK(1); time.sleep_us(5)
        SCK(0)
    time.sleep_us(10); CS_N(1)           # rising edge -> FPGA loads A, B, op
```

8 Tests Performed

8.1 Test cases

Table 7 lists the tests performed on hardware and the expected analyzer readings. The operands target edge cases: signed overflow, a borrow, the difference between logical and arithmetic shifts, and signed versus unsigned comparison.

Table 7: Test cases and expected results.

Operation	SPI bytes	Result	Zero	Carry	Overflow (LED)
ADD 7F, 01	00 7F 01	80	0	0	1
SUB 10, 20	08 10 20	F0	0	1	0
AND F0, 3C	07 F0 3C	30	0	0	0
OR F0, 0F	06 F0 0F	FF	0	0	0
XOR AA, FF	04 AA FF	55	0	0	0
SLL 81, 03	01 81 03	08	0	0	0
SRL 81, 03	05 81 03	10	0	0	0
SRA 81, 03	0D 81 03	F0	0	0	0
SLT FF, 01	02 FF 01	01	0	0	0
SLTU FF, 01	03 FF 01	00	1	0	0
SLL 0D, 02	01 0D 02	34	0	0	0

8.2 Analyzer setup

In WaveForms' Logic tool, DIO 0–7 were grouped as a hexadecimal bus, DIO 8 and DIO 9 shown as the zero and carry flags, and an SPI decoder configured on DIO 11 (clock), DIO 12 (data) and DIO 13 (select, active low), sampling on the rising edge, 8 bits, MSB first. The trigger was set to the falling edge of DIO 13 (start of frame). The full settings are listed in Appendix G.

8.3 Results of the eleven test cases

All eleven tests produced the expected results on the analyzer, with the overflow LED lighting only for ADD 7F, 01. Out of these eleven tests, the carry flag was high only for SUB 10, 20 and the zero flag only for SLTU FF, 01, as expected.

8.4 Regular versus shared ALU on hardware

Both bitstreams were loaded in turn with identical wiring and test sequence; the analyzer showed identical results for all eleven test cases.

9 Resource Utilisation

Both designs were synthesised and implemented in Go Configure with identical interface logic, pin assignment and clocking; only the ALU core differs. Table 8 lists the figures from the Resources Report.

Table 8: Resource comparison from the Go Configure Resources Report (full design).

Resource	Regular	Shared	Change
CLB LUT5s (of 1,120)	155 (13.84%)	129 (11.52%)	−26 (−16.8%)
CLBs (of 140)	30 (21.43%)	27 (19.29%)	−3 (−10%)
Flip-flops	60	60	unchanged
CLB FFs	46	46	unchanged
IOB FFs	14	14	unchanged
GPIOs (of 19)	14	14	unchanged
OSCs	1	1	unchanged

Sharing one adder and one shifter removes 26 LUTs, 16.8% of the complete design. Because the serial loader, output registers and pin logic are identical in both designs, the whole saving comes from the ALU core, so the reduction within the core itself is larger than 16.8%. The flip-flop count is unchanged, as expected: the ALU core is purely combinational, and the change affects only combinational logic.

10 Conclusion

An 8-bit ALU implementing all ten RV32I integer operations was designed, implemented on the Shrike-Lite board and verified on hardware. A serial input interface solved the board's pin limitation and allowed the protocol analyzer to display each operation's inputs alongside its result. The resource-shared ALU, built from a single adder and a single shifter, produced results identical to the regular design in every hardware test while using 26 fewer LUTs, 16.8% of the complete design, according to the Go Configure Resources Report. A slide-switch interface allows any operation and operands to be entered and verified on the analyzer.

11 Proposed Evaluation Scheme

The 18 marks for the demonstration are proposed to be distributed equally between two parts, as shown in Table 9.

Table 9: Proposed distribution of the demonstration marks.

Part	Demonstration	Marks
1	Any one of the ten operations, chosen by the student	9
2	One operation chosen by the examiner	9
	Total	**18**

In both parts, an operation counts as demonstrated when its operands and opcode are set on the slide switches and the result captured on the protocol analyzer matches the expected value, together with the corresponding status flags.

References

[1] A. Waterman and K. Asanović (eds.), *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Document Version 20191213, RISC-V Foundation, 2019.

[2] S. R. Sarangi, *Basic Computer Architecture*, Version 3.09. Freely available online; print edition: White Falcon Publishing, 2021. Available: `https://www.cse.iitd.ac.in/~srsarangi/archbooksoft.html`

[3] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware/Software Interface*, 2nd ed., Morgan Kaufmann, 2020.

[4] S. L. Harris and D. M. Harris, *Digital Design and Computer Architecture: RISC-V Edition*, Morgan Kaufmann, 2021.

[5] Renesas Electronics, *SLG47910 ForgeFPGA Datasheet*. Available: `https://www.renesas.com`

[6] Renesas Electronics, *Go Configure Software Hub*. Available: `https://www.renesas.com`

[7] Vicharak, *Shrike: documentation and examples*. Available: `https://github.com/vicharak-in/shrike`

[8] Raspberry Pi Ltd, *RP2040 Datasheet*. Available: `https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf`

[9] MicroPython project, *MicroPython documentation*. Available: `https://docs.micropython.org`

[10] Thonny, *Thonny: Python IDE for beginners*. Available: `https://thonny.org`

[11] Digilent, *Analog Discovery 3 and WaveForms reference manuals*. Available: `https://digilent.com/reference`

[12] Analyze My Writing, *What is a Readability Index?* Available: `https://www.analyzemywriting.com/readability_indices.html`

[13] textstat contributors, *textstat: Python package to calculate readability statistics of a text object*, version 0.7.13. Available: `https://github.com/textstat/textstat`

Glossary

ALU Arithmetic logic unit: the part of a processor that performs arithmetic and logical operations.

bitstream The configuration file that programs an FPGA.

CLB Configurable logic block: a group of LUTs and flip-flops in the FPGA fabric.

CS_N Active-low chip select: frames an SPI transfer; its rising edge marks the end of a frame.

FPGA Field-programmable gate array: a chip whose logic is configured after manufacture by loading a bitstream.

generate In an adder, the signal $g_i = a_i \wedge b_i$ indicating that bit i produces a carry by itself.

GPIO General-purpose input/output: a pin that can be configured as an input or an output.

IOB Input/output block: the circuitry connecting an FPGA pin to the internal logic.

ISA Instruction set architecture: the instructions and encodings a processor understands.

LSB Least significant bit.

LUT Look-up table: the basic logic element of an FPGA; a LUT5 implements any Boolean function of five inputs.

MSB Most significant bit.

OE Output enable: the signal that switches on a pin's output driver.

overflow A signed result that does not fit in the available bits, e.g. two positive numbers giving a negative sum.

PMOD A 12-pin peripheral header standard; on the Shrike-Lite it carries FPGA_IO8–IO15.

propagate In an adder, the signal $p_i = a_i \oplus b_i$ indicating that bit i passes an incoming carry on.

readability index A formula estimating the years of education needed to read a text easily.

RTL Register-transfer level: a hardware description in terms of registers and the logic between them.

RV32I The 32-bit base integer instruction set of RISC-V.

SCK Serial clock line of an SPI link.

SPI Serial Peripheral Interface: a synchronous serial protocol with clock, data and chip-select lines.

two's complement The standard representation of signed binary numbers, in which the negative of a number is formed by inverting all its bits and adding 1.

A Plagiarism Report

This report was checked for plagiarism on turnitin.com. The summary below matches the submission made there; the full Turnitin report follows.

Table 10: Turnitin plagiarism check.

Item	Value
Submission ID	
Date of check	
Overall similarity	

turnitin Page 24 of 39 - Integrity Submission Submission ID trn:oid:::1:3710129477

B Reading Indices

The readability of this report was assessed with the readability indexs described by Analyze My Writing [12]. Each index estimates the (United States) grade level, i.e. the number of years of formal education, needed to read a text without difficulty; the lower the index, the easier the text.

B.1 Method

Only the running text was analysed: the abstract and the prose of Sections 1–11, including Sections 1.1 and 1.2. The front matter, tables, figures, code listings, equations, appendices and references were excluded, since they consist mainly of symbols, numbers and code rather than sentences. The prose was extracted into a plain-text file and analysed with *textstat* (version 0.7.13) [13], an open-source Python library implementing the standard formulas; the script used, `readability_textstat.py`, is available in the project repository.

B.2 Results

Table 11: Reading indices of the report's running text (textstat 0.7.13).

Measure	Value
Sentences	83
Words	1,740
Average words per sentence	21.0
Gunning Fog Index	14.20
Flesch–Kincaid Grade Level	11.68
SMOG Index	13.62
Coleman–Liau Index	11.06
Automated Readability Index	12.86
Average grade level	12.69
Median grade level	12.86
Flesch Reading Ease (0–100, higher is easier)	48.6

B.3 Interpretation

The five grade-level indices range from 11.1 to 14.2, with an average of 12.7 and a median of 12.9. On the scale given by Analyze My Writing [12], this corresponds to the "Undergraduate" reading level (12–16), the level of college textbooks; values above 12 are described as a thoroughly adult level, and technical and professional literature generally scores above 12. The Flesch Reading Ease of 48.6 agrees, placing the text in the "difficult" band (30–50) typical of university-level writing. The scores are driven mainly by unavoidable technical vocabulary (for example *synchroniser*, *combinational* and *oscillator*) rather than by sentence length, since the average sentence of 21.0 words is moderate. The report is therefore at an appropriate level for its audience of engineering students.

As noted by Analyze My Writing, syllable-based indices are estimates, so other tools may give slightly different values.

C Worked Examples of the Shared ALU

The following examples show how the shared adder and shifter of Section 6 produce the result of each test case in Table 7.

ADD 7F + 01. With sub = 0: $p = 0111\,1110$, $g = 0000\,0001$. The carry generated in bit 0 propagates through bits 1–6, so $s = 1000\,0000 = 80$, $c_8 = 0$ and $c_7 = 1$, giving overflow $V = c_8 \oplus c_7 = 1$: two positive numbers produced a negative result.

SUB 10 − 20. With sub = 1, B is inverted to 1101 1111 and $c_0 = 1$. The adder produces $s = 1111\,0000 = \mathrm{F0}$ with $c_8 = 0$, so the borrow (carry flag) is 1.

SLT and SLTU on FF, 01. Both compute $\mathrm{FF} - 01 = \mathrm{FE}$. Signed, $s_7 = 1$ and $V = 0$, so SLT = 1 ($-1 < 1$). Unsigned, $c_8 = 1$ (no borrow), so SLTU = 0 ($255 \not< 1$). One subtraction answers both questions.

SLL 0D by 2. Mirror: 0000 1101 → 1011 0000; shift right by 2: 0010 1100; mirror back: 0011 0100 = 34, which equals 0D ≪ 2.

AND F0, 3C. With sub = 0 the adder forms $p = 1111\,0000 \oplus 0011\,1100 = 1100\,1100$ and $g = 1111\,0000 \wedge 0011\,1100 = 0011\,0000$. The AND result is taken directly from the generate signals: $g = 30$. The carry chain is not used.

OR F0, 0F. Here $p = 1111\,1111$ and $g = 0000\,0000$, because the operands share no 1-bits. The OR result is $p \vee g = 1111\,1111 = \mathrm{FF}$. Since $g = 0$, OR and XOR coincide for these operands.

XOR AA, FF. The XOR result is the propagate signal itself: $p = 1010\,1010 \oplus 1111\,1111 = 0101\,0101 = 55$, i.e. every bit of A inverted.

SLL 81 by 3. Mirror: 1000 0001 → 1000 0001 (this pattern reads the same in both directions); shift right by 3 with zero fill: 0001 0000; mirror back: 0000 1000 = 08, which equals 81 ≪ 3 truncated to 8 bits.

SRL 81 by 3. The shifter fills with 0: 1000 0001 → 0001 0000 = 10.

SRA 81 by 3. The same shifter fills with the sign bit (1): 1000 0001 → 1111 0000 = F0. Read as signed numbers, 81 = −127 and F0 = −16: an arithmetic right shift divides by 2^3 and rounds towards $-\infty$ ($-127/8 = -15.875 \to -16$), whereas SRL on the same bits gives 10.

D Source Code

The complete sources are also available in the project repository: `https://github.com/samairakhan2806/ELL305-Project-SAMAIRA-KHAN-2024EE10201/tree/main`

D.1 Resource-shared ALU

Listing 2: Resource-shared ALU (shared adder and shifter); `/hardware/main_shared.v` in the project repository.

```
module shared_adder (
    input  [7:0] a,
    input  [7:0] b,
    input        sub,          // 1 = compute a - b
    output [7:0] sum,
    output [7:0] p,            // propagate = a ^ b'  (b' = b, or ~b when sub)
    output [7:0] g,            // generate  = a & b'
    output       cout,         // carry out of bit 7
    output       overflow      // signed overflow
);
    wire [7:0] bx = b ^ {8{sub}};      // invert b for subtraction
    wire [8:0] c;                      // carry chain

    assign p    = a ^ bx;
    assign g    = a & bx;
    assign c[0] = sub;                 // +1 completes two's complement

    genvar i;
    generate
        for (i = 0; i < 8; i = i + 1) begin : carry_chain
            assign c[i+1] = g[i] | (p[i] & c[i]);
        end
    endgenerate

    assign sum      = p ^ c[7:0];
    assign cout     = c[8];
    assign overflow = c[8] ^ c[7];     // carry into MSB != carry out of MSB
endmodule

module shared_shifter (
    input  [7:0] a,
    input  [2:0] shamt,
    input        left,         // 1 = SLL (mirror in, mirror out)
    input        arith,        // 1 = SRA (fill with sign bit)
    output [7:0] y
);
    function [7:0] reverse8(input [7:0] x);
        integer k;
        for (k = 0; k < 8; k = k + 1) reverse8[k] = x[7-k];
    endfunction

    wire [7:0] x    = left ? reverse8(a) : a;
    wire       fill = arith & a[7];
    wire [8:0] r    = $signed({fill, x}) >>> shamt;   // 9-bit: fill bit on top

    assign y = left ? reverse8(r[7:0]) : r[7:0];
```

```
endmodule

module alu_8bit (
    input       [7:0] a,
    input       [7:0] b,
    input       [3:0] op,
    output reg  [7:0] y,
    output            zero,
    output reg        carry,     // ADD: carry out, SUB: borrow, else 0
    output reg        overflow   // ADD/SUB signed overflow, else 0
);
    localparam OP_ADD  = 4'b0000, OP_SLL = 4'b0001, OP_SLT = 4'b0010,
               OP_SLTU = 4'b0011, OP_XOR = 4'b0100, OP_SRL = 4'b0101,
               OP_OR   = 4'b0110, OP_AND = 4'b0111, OP_SUB = 4'b1000,
               OP_SRA  = 4'b1101;

    // SUB, SLT and SLTU all need a - b
    wire sub = (op == OP_SUB) || (op == OP_SLT) || (op == OP_SLTU);

    wire [7:0] sum, p, g, sh;
    wire       cout, v;

    shared_adder u_add (
        .a(a), .b(b), .sub(sub),
        .sum(sum), .p(p), .g(g), .cout(cout), .overflow(v)
    );

    shared_shifter u_shift (
        .a(a), .shamt(b[2:0]),
        .left(op == OP_SLL), .arith(op == OP_SRA),
        .y(sh)
    );

    wire lt_signed   = sum[7] ^ v;     // N xor V
    wire lt_unsigned = ~cout;          // borrow

    always @(*) begin
        y        = 8'h00;
        carry    = 1'b0;
        overflow = 1'b0;
        case (op)
            OP_ADD:  begin y = sum; carry = cout;  overflow = v; end
            OP_SUB:  begin y = sum; carry = ~cout; overflow = v; end
            OP_SLT:  y = {7'b0, lt_signed};
            OP_SLTU: y = {7'b0, lt_unsigned};
            OP_XOR:  y = p;
            OP_AND:  y = g;
            OP_OR:   y = p | g;
            OP_SLL, OP_SRL, OP_SRA: y = sh;
            default: y = 8'h00;
        endcase
    end

    assign zero = (y == 8'h00);
endmodule

module serial_loader (
    input            clk,
```

```
    input            sck_pin,
    input            sdi_pin,
    input            cs_n_pin,
    output reg [7:0] a,
    output reg [7:0] b,
    output reg [3:0] op,
    output reg       load_pulse      // one clk high when a new frame arrives
);
    reg [2:0]  sck_s;
    reg [2:0]  cs_s;
    reg [1:0]  sdi_s;
    reg [23:0] shreg;

    always @(posedge clk) begin
        sck_s <= {sck_s[1:0], sck_pin};
        cs_s  <= {cs_s[1:0],  cs_n_pin};
        sdi_s <= {sdi_s[0],   sdi_pin};
        load_pulse <= 1'b0;

        // shift one bit on each SCK rising edge while CS_N is low
        if (!cs_s[1] && (sck_s[2:1] == 2'b01))
            shreg <= {shreg[22:0], sdi_s[1]};

        // CS_N rising edge = end of frame -> load operands
        if (cs_s[2:1] == 2'b01) begin
            op         <= shreg[19:16];
            a          <= shreg[15:8];
            b          <= shreg[7:0];
            load_pulse <= 1'b1;
        end
    end
endmodule

(* top *)
module alu_top (
    (* iopad_external_pin, clkbuf_inhibit *) input clk,     // internal OSC clock
    (* iopad_external_pin *) output       clk_en,           // OSC enable

    // inputs from RP2040 (SPI-style frame)
    (* iopad_external_pin *) input        spi_sck,          // FPGA_IO18
    (* iopad_external_pin *) input        spi_sdi,          // FPGA_IO7  (moved from
    FPGA_IO17, see README)
    (* iopad_external_pin *) input        spi_cs_n,         // FPGA_IO0

    // outputs to protocol analyzer
    (* iopad_external_pin *) output [7:0] result,          // FPGA_IO8..IO15 (PMOD)
    (* iopad_external_pin *) output [7:0] result_oe,
    (* iopad_external_pin *) output       zero,             // FPGA_IO1
    (* iopad_external_pin *) output       zero_oe,
    (* iopad_external_pin *) output       carry,            // FPGA_IO2
    (* iopad_external_pin *) output       carry_oe,
    (* iopad_external_pin *) output       overflow,         // FPGA_IO16 = on-board LED
    (* iopad_external_pin *) output       overflow_oe
);
    assign clk_en      = 1'b1;
    assign result_oe   = 8'hFF;
    assign zero_oe     = 1'b1;
    assign carry_oe    = 1'b1;
```

```
162     assign overflow_oe = 1'b1;
163
164     wire [7:0] a, b, y;
165     wire [3:0] op;
166     wire       load_pulse, z, c, v;
167
168     serial_loader u_rx (
169         .clk(clk), .sck_pin(spi_sck), .sdi_pin(spi_sdi), .cs_n_pin(spi_cs_n),
170         .a(a), .b(b), .op(op), .load_pulse(load_pulse)
171     );
172
173     alu_8bit u_alu (
174         .a(a), .b(b), .op(op),
175         .y(y), .zero(z), .carry(c), .overflow(v)
176     );
177
178     // register the outputs so all 11 lines change on the same clock edge
179     // (one clean transition on the analyzer instead of glitches)
180     reg [7:0] result_q;
181     reg       zero_q, carry_q, overflow_q;
182     always @(posedge clk) begin
183         result_q   <= y;
184         zero_q     <= z;
185         carry_q    <= c;
186         overflow_q <= v;
187     end
188     assign result   = result_q;
189     assign zero     = zero_q;
190     assign carry    = carry_q;
191     assign overflow = overflow_q;
192
193 endmodule
```

D.2 Regular ALU

Listing 3: Regular ALU; `/hardware/main.v` in the project repository.

```
1   module alu_add (
2       input  [7:0] a,
3       input  [7:0] b,
4       output [7:0] y,
5       output       carry,      // unsigned carry out of bit 7
6       output       overflow    // signed overflow
7   );
8       wire [8:0] sum = {1'b0, a} + {1'b0, b};
9       assign y        = sum[7:0];
10      assign carry    = sum[8];
11      assign overflow = (a[7] == b[7]) && (y[7] != a[7]);
12  endmodule
13
14  module alu_sub (
15      input  [7:0] a,
16      input  [7:0] b,
17      output [7:0] y,
18      output       borrow,     // 1 when a < b (unsigned)
19      output       overflow    // signed overflow
20  );
```

```
    wire [8:0] diff = {1'b0, a} - {1'b0, b};
    assign y        = diff[7:0];
    assign borrow   = diff[8];
    assign overflow = (a[7] != b[7]) && (y[7] != a[7]);
endmodule

module alu_and (input [7:0] a, input [7:0] b, output [7:0] y);
    assign y = a & b;
endmodule

module alu_or (input [7:0] a, input [7:0] b, output [7:0] y);
    assign y = a | b;
endmodule

module alu_xor (input [7:0] a, input [7:0] b, output [7:0] y);
    assign y = a ^ b;
endmodule

module alu_sll (input [7:0] a, input [2:0] shamt, output [7:0] y);
    assign y = a << shamt;                          // logical left
endmodule

module alu_srl (input [7:0] a, input [2:0] shamt, output [7:0] y);
    assign y = a >> shamt;                          // logical right (zero fill)
endmodule

module alu_sra (input [7:0] a, input [2:0] shamt, output [7:0] y);
    assign y = $signed(a) >>> shamt;                // arithmetic right (sign fill)
endmodule

module alu_slt (input [7:0] a, input [7:0] b, output [7:0] y);
    assign y = {7'b0, ($signed(a) < $signed(b))};  // signed compare
endmodule

module alu_sltu (input [7:0] a, input [7:0] b, output [7:0] y);
    assign y = {7'b0, (a < b)};                     // unsigned compare
endmodule

module alu_8bit (
    input       [7:0] a,
    input       [7:0] b,
    input       [3:0] op,
    output reg  [7:0] y,
    output            zero,
    output reg        carry,      // ADD: carry out, SUB: borrow, else 0
    output reg        overflow    // ADD/SUB signed overflow, else 0
);
    localparam OP_ADD  = 4'b0000, OP_SLL = 4'b0001, OP_SLT = 4'b0010,
               OP_SLTU = 4'b0011, OP_XOR = 4'b0100, OP_SRL = 4'b0101,
               OP_OR   = 4'b0110, OP_AND = 4'b0111, OP_SUB = 4'b1000,
               OP_SRA  = 4'b1101;

    wire [7:0] y_add, y_sub, y_and, y_or, y_xor;
    wire [7:0] y_sll, y_srl, y_sra, y_slt, y_sltu;
    wire       c_add, v_add, c_sub, v_sub;

    alu_add  u_add  (.a(a), .b(b), .y(y_add), .carry(c_add), .overflow(v_add));
    alu_sub  u_sub  (.a(a), .b(b), .y(y_sub), .borrow(c_sub), .overflow(v_sub));
```

```
    alu_and  u_and  (.a(a), .b(b), .y(y_and));
    alu_or   u_or   (.a(a), .b(b), .y(y_or));
    alu_xor  u_xor  (.a(a), .b(b), .y(y_xor));
    alu_sll  u_sll  (.a(a), .shamt(b[2:0]), .y(y_sll));
    alu_srl  u_srl  (.a(a), .shamt(b[2:0]), .y(y_srl));
    alu_sra  u_sra  (.a(a), .shamt(b[2:0]), .y(y_sra));
    alu_slt  u_slt  (.a(a), .b(b), .y(y_slt));
    alu_sltu u_sltu (.a(a), .b(b), .y(y_sltu));

    always @(*) begin
        y        = 8'h00;
        carry    = 1'b0;
        overflow = 1'b0;
        case (op)
            OP_ADD:  begin y = y_add; carry = c_add; overflow = v_add; end
            OP_SUB:  begin y = y_sub; carry = c_sub; overflow = v_sub; end
            OP_AND:  y = y_and;
            OP_OR:   y = y_or;
            OP_XOR:  y = y_xor;
            OP_SLL:  y = y_sll;
            OP_SRL:  y = y_srl;
            OP_SRA:  y = y_sra;
            OP_SLT:  y = y_slt;
            OP_SLTU: y = y_sltu;
            default: y = 8'h00;
        endcase
    end

    assign zero = (y == 8'h00);
endmodule

module serial_loader (
    input            clk,
    input            sck_pin,
    input            sdi_pin,
    input            cs_n_pin,
    output reg [7:0] a,
    output reg [7:0] b,
    output reg [3:0] op,
    output reg       load_pulse      // one clk high when a new frame arrives
);
    reg [2:0]  sck_s;
    reg [2:0]  cs_s;
    reg [1:0]  sdi_s;
    reg [23:0] shreg;

    always @(posedge clk) begin
        sck_s <= {sck_s[1:0], sck_pin};
        cs_s  <= {cs_s[1:0],  cs_n_pin};
        sdi_s <= {sdi_s[0],   sdi_pin};
        load_pulse <= 1'b0;

        if (!cs_s[1] && (sck_s[2:1] == 2'b01))
            shreg <= {shreg[22:0], sdi_s[1]};

        if (cs_s[2:1] == 2'b01) begin
            op         <= shreg[19:16];
            a          <= shreg[15:8];
```

```
            b          <= shreg[7:0];
            load_pulse <= 1'b1;
        end
    end
endmodule

(* top *)
module alu_top (
    (* iopad_external_pin, clkbuf_inhibit *) input clk,     // internal OSC clock
    (* iopad_external_pin *) output        clk_en,          // OSC enable

    // inputs from RP2040 (SPI-style frame)
    (* iopad_external_pin *) input         spi_sck,         // FPGA_IO18
    (* iopad_external_pin *) input         spi_sdi,         // FPGA_IO7  (moved from
    FPGA_IO17, see README)
    (* iopad_external_pin *) input         spi_cs_n,        // FPGA_IO0

    // outputs to protocol analyzer
    (* iopad_external_pin *) output [7:0] result,          // FPGA_IO8..IO15 (PMOD)
    (* iopad_external_pin *) output [7:0] result_oe,
    (* iopad_external_pin *) output        zero,            // FPGA_IO1
    (* iopad_external_pin *) output        zero_oe,
    (* iopad_external_pin *) output        carry,           // FPGA_IO2
    (* iopad_external_pin *) output        carry_oe,
    (* iopad_external_pin *) output        overflow,        // FPGA_IO16 = on-board LED
    (* iopad_external_pin *) output        overflow_oe
);
    assign clk_en      = 1'b1;
    assign result_oe   = 8'hFF;
    assign zero_oe     = 1'b1;
    assign carry_oe    = 1'b1;
    assign overflow_oe = 1'b1;

    wire [7:0] a, b, y;
    wire [3:0] op;
    wire       load_pulse, z, c, v;

    serial_loader u_rx (
        .clk(clk), .sck_pin(spi_sck), .sdi_pin(spi_sdi), .cs_n_pin(spi_cs_n),
        .a(a), .b(b), .op(op), .load_pulse(load_pulse)
    );

    alu_8bit u_alu (
        .a(a), .b(b), .op(op),
        .y(y), .zero(z), .carry(c), .overflow(v)
    );

    reg [7:0] result_q;
    reg       zero_q, carry_q, overflow_q;
    always @(posedge clk) begin
        result_q   <= y;
        zero_q     <= z;
        carry_q    <= c;
        overflow_q <= v;
    end
    assign result   = result_q;
    assign zero     = zero_q;
    assign carry    = carry_q;
```

```
    assign overflow = overflow_q;

endmodule
```

D.3 RP2040 firmware

Listing 4: RP2040 firmware; `/firmware/alu_test.py` in the project repository.

```
import shrike, time
from machine import Pin

shrike.flash("alu.bin")
time.sleep_ms(100)

SCK  = Pin(14, Pin.OUT, value=0)
SDI  = Pin(17, Pin.OUT, value=0)
CS_N = Pin(16, Pin.OUT, value=1)

OPS = {"ADD": 0b0000, "SLL": 0b0001, "SLT": 0b0010, "SLTU": 0b0011,
       "XOR": 0b0100, "SRL": 0b0101, "OR":  0b0110, "AND":  0b0111,
       "SUB": 0b1000, "SRA": 0b1101}

def send(op, a, b):
    """Send one 24-bit frame: {0000, op} A B, MSB first, SPI mode 0."""
    frame = ((op & 0xF) << 16) | ((a & 0xFF) << 8) | (b & 0xFF)
    CS_N(0); time.sleep_us(10)
    for i in range(23, -1, -1):
        SDI((frame >> i) & 1); time.sleep_us(5)
        SCK(1); time.sleep_us(5)
        SCK(0)
    time.sleep_us(10); CS_N(1)              # rising edge -> FPGA loads A, B, op

def s8(x):                                  # 8-bit two's complement -> int
    return x - 256 if x & 0x80 else x

def expected(name, a, b):
    sh = b & 7
    r = {"ADD": a + b, "SUB": a - b, "AND": a & b, "OR": a | b, "XOR": a ^ b,
         "SLL": a << sh, "SRL": a >> sh, "SRA": s8(a) >> sh,
         "SLT": int(s8(a) < s8(b)), "SLTU": int(a < b)}[name]
    return r & 0xFF

def demo(gap_ms=500):
    """One frame per operation; trigger the analyzer on CS_N rising."""
    tests = [("ADD", 0x7F, 0x01), ("SUB", 0x10, 0x20), ("AND", 0xF0, 0x3C),
             ("OR", 0xF0, 0x0F), ("XOR", 0xAA, 0xFF), ("SLL", 0x81, 0x03),
             ("SRL", 0x81, 0x03), ("SRA", 0x81, 0x03), ("SLT", 0xFF, 0x01),
             ("SLTU", 0xFF, 0x01)]
    for name, a, b in tests:
        send(OPS[name], a, b)
        print("%-4s A=%02X B=%02X -> expect %02X" % (name, a, b, expected(name, a, b))
    )
        time.sleep_ms(gap_ms)

A_PINS  = [5, 6, 7, 8, 9, 10, 11, 15]       # A0..A7  (A0 = LSB)
B_PINS  = [18, 19, 20, 21, 22, 23, 24, 25] # B0..B7
OP_PINS = [26, 27, 28, 29]                  # op0..op3
```

```
49
50  SWITCH_TO_GND = True
51
52  NAMES = {v: k for k, v in OPS.items()}
53
54  def live_mode():
55      pull = Pin.PULL_UP if SWITCH_TO_GND else None
56      pins = [Pin(p, Pin.IN, pull) for p in A_PINS + B_PINS + OP_PINS]
57
58      def read_all():
59          v = 0
60          for i, p in enumerate(pins):
61              bit = p.value()
62              if SWITCH_TO_GND:
63                  bit ^= 1                    # ON pulls the pin low -> count as 1
64              v |= bit << i
65          return v
66
67      last = None
68      print("Live mode: set A, B and op on the switches; the result follows.")
69      while True:
70          v = read_all()
71          time.sleep_ms(20)
72          if v == read_all() and v != last:  # same for 20 ms = settled, and changed
73              last = v
74              a, b, op = v & 0xFF, (v >> 8) & 0xFF, (v >> 16) & 0xF
75              send(op, a, b)
76              name = NAMES.get(op, "????")
77              exp = ("%02X" % expected(name, a, b)) if name in OPS else "00 (unused op)"
78              print("%-4s A=%02X B=%02X -> expect %s" % (name, a, b, exp))
79          time.sleep_ms(10)
80
81  #while True:
82  #    demo()
83  #demo()
84  live_mode()
```

E I/O Planner Assignment

Identical for both designs. Every output port is assigned to the OUT row of its GPIO and its `_oe` port to the OE row of the same GPIO.

Table 12: Complete I/O Planner assignment.

I/O Planner row	Port	Direction
OSC_CLK	`clk`	input
OSC_EN	`clk_en`	output
GPIO0_IN	`spi_cs_n`	input
GPIO7_IN	`spi_sdi`	input
GPIO18_IN	`spi_sck`	input
GPIO1_OUT / _OE	`zero` / `zero_oe`	output
GPIO2_OUT / _OE	`carry` / `carry_oe`	output
GPIO8–GPIO15 OUT / OE	`result[0..7]` / `result_oe[0..7]`	output
GPIO16_OUT / _OE	`overflow` / `overflow_oe`	output
GPIO3–GPIO6	– (used for bitstream loading)	–
GPIO17	– (unused)	–

F Wiring

Table 13: Board jumpers.

From	To	Signal
RP_IO14 (left header, pin 11)	FPGA_IO18 (left header, pin 15)	SCK
RP_IO17 (right header, pin 17)	FPGA_IO7 (left header, pin 19)	data
RP_IO16 (right header, pin 18)	FPGA_IO0 (left header, pin 16)	CS_N

Table 14: Analog Discovery 3 connections.

Analyzer	Board pin	Signal
GND	board GND	ground
DIO 0–7	FPGA_IO8–IO15 (PMOD)	result bits 0–7
DIO 8	FPGA_IO1	zero
DIO 9	FPGA_IO2	carry
DIO 11	FPGA_IO18	SCK
DIO 12	FPGA_IO7	data
DIO 13	FPGA_IO0	CS_N

Table 15: PMOD header layout (column 1 at the 3.3 V end).

	col 1	col 2	col 3	col 4	col 5	col 6
top	3.3 V	GND	IO8	IO10	IO12	IO14
bottom	3.3 V	GND	IO9	IO11	IO13	IO15

G WaveForms Configuration

Table 16: Logic analyzer settings.

Setting	Value
Bus	DIO 0–7 (DIO 0 = LSB), hexadecimal.
Signals	DIO 8 (zero), DIO 9 (carry).
SPI decoder	Select DIO 13 (active low), Clock DIO 11, Data DIO 12; sample on rising edge, 8 bits, MSB first, hexadecimal.
Trigger	Normal mode, falling edge of DIO 13.
Sample rate	1 MHz.
Time base	about 200 µs/div for a single frame.

Index

ALU, 1
Analog Discovery 3, 2

bit reversal, 5

carry chain, 5

firmware, 6
ForgeFPGA, 2

generate signal, 5
Go Configure, 2

instruction set architecture, 1

LUT, 8

opcode, 2
output enable, 4
overflow, 1, 3

pin assignment, 4
propagate signal, 5

readability indices, 13
reference model, 6
regular ALU, 4
resource utilisation, 7
resource-shared ALU, 4
RISC-V, 1
RP2040, 2

serial frame, 3
serial loader, 4
shifter, 5
Shrike-Lite, 2
slide switches, 5
SPI, 3
status flags, 3

test cases, 6
two's complement, 5

Verilog, 1

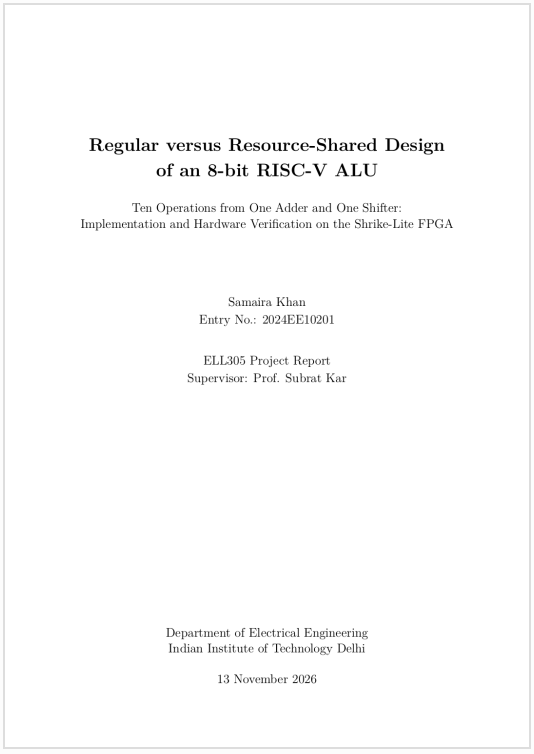
WaveForms, 2

Digital Receipt

This receipt acknowledges that Turnitin received your paper. Below you will find the receipt information regarding your submission.

The first page of your submissions is displayed below.

Submission author:	SAMAIRA KHAN
Assignment title:	Project
Submission title:	Project_2024EE10201_Samaira_Khan.pdf
File name:	Project_2024EE10201_Samaira_Khan.pdf
File size:	428.25K
Page count:	34
Word count:	8,053
Character count:	37,008
Submission date:	13-Nov-2026 01:16PM (UTC+0530)
Submission ID:	3088253470



Regular versus Resource-Shared Design of an 8-bit RISC-V ALU

Ten Operations from One Adder and One Shifter:
Implementation and Hardware Verification on the Shrike-Lite FPGA

Samaira Khan
Entry No.: 2024EE10201

ELL305 Project Report
Supervisor: Prof. Subrat Kar

Department of Electrical Engineering
Indian Institute of Technology Delhi

13 November 2026

Copyright 2026 Turnitin. All rights reserved.

B Reading Indices

The readability of this report was assessed with the readability indexs described by Analyze My Writing [12]. Each index estimates the (United States) grade level, i.e. the number of years of formal education, needed to read a text without difficulty; the lower the index, the easier the text.

B.1 Method

Only the running text was analysed: the abstract and the prose of Sections 1–11, including Sections 1.1 and 1.2. The front matter, tables, figures, code listings, equations, appendices and references were excluded, since they consist mainly of symbols, numbers and code rather than sentences. The prose was extracted into a plain-text file and analysed with *textstat* (version 0.7.13) [13], an open-source Python library implementing the standard formulas; the script used, `readability_textstat.py`, is available in the project repository.

B.2 Results

Table 11: Reading indices of the report's running text (textstat 0.7.13).

Measure	**Value**
Sentences	83
Words	1,740
Average words per sentence	21.0
Gunning Fog Index	14.20
Flesch–Kincaid Grade Level	11.68
SMOG Index	13.62
Coleman–Liau Index	11.06
Automated Readability Index	12.86
Average grade level	12.69
Median grade level	12.86
Flesch Reading Ease (0–100, higher is easier)	48.6

B.3 Interpretation

The five grade-level indices range from 11.1 to 14.2, with an average of 12.7 and a median of 12.9. On the scale given by Analyze My Writing [12], this corresponds to the "Undergraduate" reading level (12–16), the level of college textbooks; values above 12 are described as a thoroughly adult level, and technical and professional literature generally scores above 12. The Flesch Reading Ease of 48.6 agrees, placing the text in the "difficult" band (30–50) typical of university-level writing. The scores are driven mainly by unavoidable technical vocabulary (for example *synchroniser*, *combinational* and *oscillator*) rather than by sentence length, since the average sentence of 21.0 words is moderate. The report is therefore at an appropriate level for its audience of engineering students.

As noted by Analyze My Writing, syllable-based indices are estimates, so other tools may give slightly different values.

C Worked Examples of the Shared ALU

The following examples show how the shared adder and shifter of Section 6 produce the result of each test case in Table 7.

ADD 7F + 01. With sub = 0: $p = 0111\,1110$, $g = 0000\,0001$. The carry generated in bit 0 propagates through bits 1–6, so $s = 1000\,0000 = 80$, $c_8 = 0$ and $c_7 = 1$, giving overflow $V = c_8 \oplus c_7 = 1$: two positive numbers produced a negative result.

SUB 10 − 20. With sub = 1, B is inverted to 1101 1111 and $c_0 = 1$. The adder produces $s = 1111\,0000 = \mathrm{F0}$ with $c_8 = 0$, so the borrow (carry flag) is 1.

SLT and SLTU on FF, 01. Both compute $\mathrm{FF} - 01 = \mathrm{FE}$. Signed, $s_7 = 1$ and $V = 0$, so SLT = 1 ($-1 < 1$). Unsigned, $c_8 = 1$ (no borrow), so SLTU = 0 ($255 \not< 1$). One subtraction answers both questions.

SLL 0D by 2. Mirror: $0000\,1101 \rightarrow 1011\,0000$; shift right by 2: $0010\,1100$; mirror back: $0011\,0100 = 34$, which equals $0\mathrm{D} \ll 2$.

AND F0, 3C. With sub = 0 the adder forms $p = 1111\,0000 \oplus 0011\,1100 = 1100\,1100$ and $g = 1111\,0000 \wedge 0011\,1100 = 0011\,0000$. The AND result is taken directly from the generate signals: $g = 30$. The carry chain is not used.

OR F0, 0F. Here $p = 1111\,1111$ and $g = 0000\,0000$, because the operands share no 1-bits. The OR result is $p \vee g = 1111\,1111 = \mathrm{FF}$. Since $g = 0$, OR and XOR coincide for these operands.

XOR AA, FF. The XOR result is the propagate signal itself: $p = 1010\,1010 \oplus 1111\,1111 = 0101\,0101 = 55$, i.e. every bit of A inverted.

SLL 81 by 3. Mirror: $1000\,0001 \rightarrow 1000\,0001$ (this pattern reads the same in both directions); shift right by 3 with zero fill: $0001\,0000$; mirror back: $0000\,1000 = 08$, which equals $81 \ll 3$ truncated to 8 bits.

SRL 81 by 3. The shifter fills with 0: $1000\,0001 \rightarrow 0001\,0000 = 10$.

SRA 81 by 3. The same shifter fills with the sign bit (1): $1000\,0001 \rightarrow 1111\,0000 = \mathrm{F0}$. Read as signed numbers, $81 = -127$ and $\mathrm{F0} = -16$: an arithmetic right shift divides by 2^3 and rounds towards $-\infty$ ($-127/8 = -15.875 \rightarrow -16$), whereas SRL on the same bits gives 10.

D Source Code

The complete sources are also available in the project repository: `https://github.com/samairakhan2806/ELL305-Project-SAMAIRA-KHAN-2024EE10201/tree/main`

D.1 Resource-shared ALU

Listing 2: Resource-shared ALU (shared adder and shifter); `/hardware/main_shared.v` in the project repository.

```
module shared_adder (
    input  [7:0] a,
    input  [7:0] b,
    input        sub,          // 1 = compute a - b
    output [7:0] sum,
    output [7:0] p,            // propagate = a ^ b'  (b' = b, or ~b when sub)
    output [7:0] g,            // generate  = a & b'
    output       cout,         // carry out of bit 7
    output       overflow      // signed overflow
);
    wire [7:0] bx = b ^ {8{sub}};      // invert b for subtraction
    wire [8:0] c;                      // carry chain

    assign p    = a ^ bx;
    assign g    = a & bx;
    assign c[0] = sub;                 // +1 completes two's complement

    genvar i;
    generate
        for (i = 0; i < 8; i = i + 1) begin : carry_chain
            assign c[i+1] = g[i] | (p[i] & c[i]);
        end
    endgenerate

    assign sum      = p ^ c[7:0];
    assign cout     = c[8];
    assign overflow = c[8] ^ c[7];     // carry into MSB != carry out of MSB
endmodule

module shared_shifter (
    input  [7:0] a,
    input  [2:0] shamt,
    input        left,         // 1 = SLL (mirror in, mirror out)
    input        arith,        // 1 = SRA (fill with sign bit)
    output [7:0] y
);
    function [7:0] reverse8(input [7:0] x);
        integer k;
        for (k = 0; k < 8; k = k + 1) reverse8[k] = x[7-k];
    endfunction

    wire [7:0] x    = left ? reverse8(a) : a;
    wire       fill = arith & a[7];
    wire [8:0] r    = $signed({fill, x}) >>> shamt;   // 9-bit: fill bit on top

    assign y = left ? reverse8(r[7:0]) : r[7:0];
```

```
endmodule

module alu_8bit (
    input      [7:0] a,
    input      [7:0] b,
    input      [3:0] op,
    output reg [7:0] y,
    output           zero,
    output reg       carry,     // ADD: carry out, SUB: borrow, else 0
    output reg       overflow   // ADD/SUB signed overflow, else 0
);
    localparam OP_ADD  = 4'b0000, OP_SLL = 4'b0001, OP_SLT = 4'b0010,
               OP_SLTU = 4'b0011, OP_XOR = 4'b0100, OP_SRL = 4'b0101,
               OP_OR   = 4'b0110, OP_AND = 4'b0111, OP_SUB = 4'b1000,
               OP_SRA  = 4'b1101;

    // SUB, SLT and SLTU all need a - b
    wire sub = (op == OP_SUB) || (op == OP_SLT) || (op == OP_SLTU);

    wire [7:0] sum, p, g, sh;
    wire       cout, v;

    shared_adder u_add (
        .a(a), .b(b), .sub(sub),
        .sum(sum), .p(p), .g(g), .cout(cout), .overflow(v)
    );

    shared_shifter u_shift (
        .a(a), .shamt(b[2:0]),
        .left(op == OP_SLL), .arith(op == OP_SRA),
        .y(sh)
    );

    wire lt_signed   = sum[7] ^ v;     // N xor V
    wire lt_unsigned = ~cout;          // borrow

    always @(*) begin
        y        = 8'h00;
        carry    = 1'b0;
        overflow = 1'b0;
        case (op)
            OP_ADD:  begin y = sum; carry = cout;  overflow = v; end
            OP_SUB:  begin y = sum; carry = ~cout; overflow = v; end
            OP_SLT:  y = {7'b0, lt_signed};
            OP_SLTU: y = {7'b0, lt_unsigned};
            OP_XOR:  y = p;
            OP_AND:  y = g;
            OP_OR:   y = p | g;
            OP_SLL, OP_SRL, OP_SRA: y = sh;
            default: y = 8'h00;
        endcase
    end

    assign zero = (y == 8'h00);
endmodule

module serial_loader (
    input            clk,
```

```
    input            sck_pin,
    input            sdi_pin,
    input            cs_n_pin,
    output reg [7:0] a,
    output reg [7:0] b,
    output reg [3:0] op,
    output reg       load_pulse      // one clk high when a new frame arrives
);
    reg [2:0]  sck_s;
    reg [2:0]  cs_s;
    reg [1:0]  sdi_s;
    reg [23:0] shreg;

    always @(posedge clk) begin
        sck_s <= {sck_s[1:0], sck_pin};
        cs_s  <= {cs_s[1:0],  cs_n_pin};
        sdi_s <= {sdi_s[0],   sdi_pin};
        load_pulse <= 1'b0;

        // shift one bit on each SCK rising edge while CS_N is low
        if (!cs_s[1] && (sck_s[2:1] == 2'b01))
            shreg <= {shreg[22:0], sdi_s[1]};

        // CS_N rising edge = end of frame -> load operands
        if (cs_s[2:1] == 2'b01) begin
            op         <= shreg[19:16];
            a          <= shreg[15:8];
            b          <= shreg[7:0];
            load_pulse <= 1'b1;
        end
    end
endmodule

(* top *)
module alu_top (
    (* iopad_external_pin, clkbuf_inhibit *) input clk,    // internal OSC clock
    (* iopad_external_pin *) output       clk_en,          // OSC enable

    // inputs from RP2040 (SPI-style frame)
    (* iopad_external_pin *) input        spi_sck,         // FPGA_IO18
    (* iopad_external_pin *) input        spi_sdi,         // FPGA_IO7  (moved from
    FPGA_IO17, see README)
    (* iopad_external_pin *) input        spi_cs_n,        // FPGA_IO0

    // outputs to protocol analyzer
    (* iopad_external_pin *) output [7:0] result,         // FPGA_IO8..IO15 (PMOD)
    (* iopad_external_pin *) output [7:0] result_oe,
    (* iopad_external_pin *) output       zero,            // FPGA_IO1
    (* iopad_external_pin *) output       zero_oe,
    (* iopad_external_pin *) output       carry,           // FPGA_IO2
    (* iopad_external_pin *) output       carry_oe,
    (* iopad_external_pin *) output       overflow,        // FPGA_IO16 = on-board LED
    (* iopad_external_pin *) output       overflow_oe
);
    assign clk_en      = 1'b1;
    assign result_oe   = 8'hFF;
    assign zero_oe     = 1'b1;
    assign carry_oe    = 1'b1;
```

```
    assign overflow_oe = 1'b1;

    wire [7:0] a, b, y;
    wire [3:0] op;
    wire       load_pulse, z, c, v;

    serial_loader u_rx (
        .clk(clk), .sck_pin(spi_sck), .sdi_pin(spi_sdi), .cs_n_pin(spi_cs_n),
        .a(a), .b(b), .op(op), .load_pulse(load_pulse)
    );

    alu_8bit u_alu (
        .a(a), .b(b), .op(op),
        .y(y), .zero(z), .carry(c), .overflow(v)
    );

    // register the outputs so all 11 lines change on the same clock edge
    // (one clean transition on the analyzer instead of glitches)
    reg [7:0] result_q;
    reg       zero_q, carry_q, overflow_q;
    always @(posedge clk) begin
        result_q   <= y;
        zero_q     <= z;
        carry_q    <= c;
        overflow_q <= v;
    end
    assign result   = result_q;
    assign zero     = zero_q;
    assign carry    = carry_q;
    assign overflow = overflow_q;

endmodule
```

D.2 Regular ALU

Listing 3: Regular ALU; `/hardware/main.v` in the project repository.

```
module alu_add (
    input  [7:0] a,
    input  [7:0] b,
    output [7:0] y,
    output       carry,      // unsigned carry out of bit 7
    output       overflow    // signed overflow
);
    wire [8:0] sum = {1'b0, a} + {1'b0, b};
    assign y        = sum[7:0];
    assign carry    = sum[8];
    assign overflow = (a[7] == b[7]) && (y[7] != a[7]);
endmodule

module alu_sub (
    input  [7:0] a,
    input  [7:0] b,
    output [7:0] y,
    output       borrow,     // 1 when a < b (unsigned)
    output       overflow    // signed overflow
);
```

```
    wire [8:0] diff = {1'b0, a} - {1'b0, b};
    assign y        = diff[7:0];
    assign borrow   = diff[8];
    assign overflow = (a[7] != b[7]) && (y[7] != a[7]);
endmodule

module alu_and (input [7:0] a, input [7:0] b, output [7:0] y);
    assign y = a & b;
endmodule

module alu_or (input [7:0] a, input [7:0] b, output [7:0] y);
    assign y = a | b;
endmodule

module alu_xor (input [7:0] a, input [7:0] b, output [7:0] y);
    assign y = a ^ b;
endmodule

module alu_sll (input [7:0] a, input [2:0] shamt, output [7:0] y);
    assign y = a << shamt;                      // logical left
endmodule

module alu_srl (input [7:0] a, input [2:0] shamt, output [7:0] y);
    assign y = a >> shamt;                      // logical right (zero fill)
endmodule

module alu_sra (input [7:0] a, input [2:0] shamt, output [7:0] y);
    assign y = $signed(a) >>> shamt;            // arithmetic right (sign fill)
endmodule

module alu_slt (input [7:0] a, input [7:0] b, output [7:0] y);
    assign y = {7'b0, ($signed(a) < $signed(b))};  // signed compare
endmodule

module alu_sltu (input [7:0] a, input [7:0] b, output [7:0] y);
    assign y = {7'b0, (a < b)};                 // unsigned compare
endmodule

module alu_8bit (
    input      [7:0] a,
    input      [7:0] b,
    input      [3:0] op,
    output reg [7:0] y,
    output           zero,
    output reg       carry,      // ADD: carry out, SUB: borrow, else 0
    output reg       overflow    // ADD/SUB signed overflow, else 0
);
    localparam OP_ADD  = 4'b0000, OP_SLL = 4'b0001, OP_SLT = 4'b0010,
               OP_SLTU = 4'b0011, OP_XOR = 4'b0100, OP_SRL = 4'b0101,
               OP_OR   = 4'b0110, OP_AND = 4'b0111, OP_SUB = 4'b1000,
               OP_SRA  = 4'b1101;

    wire [7:0] y_add, y_sub, y_and, y_or, y_xor;
    wire [7:0] y_sll, y_srl, y_sra, y_slt, y_sltu;
    wire       c_add, v_add, c_sub, v_sub;

    alu_add  u_add  (.a(a), .b(b), .y(y_add), .carry(c_add), .overflow(v_add));
    alu_sub  u_sub  (.a(a), .b(b), .y(y_sub), .borrow(c_sub), .overflow(v_sub));
```

```
    alu_and  u_and  (.a(a), .b(b), .y(y_and));
    alu_or   u_or   (.a(a), .b(b), .y(y_or));
    alu_xor  u_xor  (.a(a), .b(b), .y(y_xor));
    alu_sll  u_sll  (.a(a), .shamt(b[2:0]), .y(y_sll));
    alu_srl  u_srl  (.a(a), .shamt(b[2:0]), .y(y_srl));
    alu_sra  u_sra  (.a(a), .shamt(b[2:0]), .y(y_sra));
    alu_slt  u_slt  (.a(a), .b(b), .y(y_slt));
    alu_sltu u_sltu (.a(a), .b(b), .y(y_sltu));

    always @(*) begin
        y        = 8'h00;
        carry    = 1'b0;
        overflow = 1'b0;
        case (op)
            OP_ADD:  begin y = y_add; carry = c_add; overflow = v_add; end
            OP_SUB:  begin y = y_sub; carry = c_sub; overflow = v_sub; end
            OP_AND:  y = y_and;
            OP_OR:   y = y_or;
            OP_XOR:  y = y_xor;
            OP_SLL:  y = y_sll;
            OP_SRL:  y = y_srl;
            OP_SRA:  y = y_sra;
            OP_SLT:  y = y_slt;
            OP_SLTU: y = y_sltu;
            default: y = 8'h00;
        endcase
    end

    assign zero = (y == 8'h00);
endmodule

module serial_loader (
    input            clk,
    input            sck_pin,
    input            sdi_pin,
    input            cs_n_pin,
    output reg [7:0] a,
    output reg [7:0] b,
    output reg [3:0] op,
    output reg       load_pulse      // one clk high when a new frame arrives
);
    reg [2:0]  sck_s;
    reg [2:0]  cs_s;
    reg [1:0]  sdi_s;
    reg [23:0] shreg;

    always @(posedge clk) begin
        sck_s <= {sck_s[1:0], sck_pin};
        cs_s  <= {cs_s[1:0],  cs_n_pin};
        sdi_s <= {sdi_s[0],   sdi_pin};
        load_pulse <= 1'b0;

        if (!cs_s[1] && (sck_s[2:1] == 2'b01))
            shreg <= {shreg[22:0], sdi_s[1]};

        if (cs_s[2:1] == 2'b01) begin
            op         <= shreg[19:16];
            a          <= shreg[15:8];
```

```
            b          <= shreg[7:0];
            load_pulse <= 1'b1;
        end
    end
endmodule

(* top *)
module alu_top (
    (* iopad_external_pin, clkbuf_inhibit *) input clk,    // internal OSC clock
    (* iopad_external_pin *) output       clk_en,          // OSC enable

    // inputs from RP2040 (SPI-style frame)
    (* iopad_external_pin *) input        spi_sck,         // FPGA_IO18
    (* iopad_external_pin *) input        spi_sdi,         // FPGA_IO7  (moved from
    FPGA_IO17, see README)
    (* iopad_external_pin *) input        spi_cs_n,        // FPGA_IO0

    // outputs to protocol analyzer
    (* iopad_external_pin *) output [7:0] result,         // FPGA_IO8..IO15 (PMOD)
    (* iopad_external_pin *) output [7:0] result_oe,
    (* iopad_external_pin *) output       zero,            // FPGA_IO1
    (* iopad_external_pin *) output       zero_oe,
    (* iopad_external_pin *) output       carry,           // FPGA_IO2
    (* iopad_external_pin *) output       carry_oe,
    (* iopad_external_pin *) output       overflow,        // FPGA_IO16 = on-board LED
    (* iopad_external_pin *) output       overflow_oe
);
    assign clk_en      = 1'b1;
    assign result_oe   = 8'hFF;
    assign zero_oe     = 1'b1;
    assign carry_oe    = 1'b1;
    assign overflow_oe = 1'b1;

    wire [7:0] a, b, y;
    wire [3:0] op;
    wire       load_pulse, z, c, v;

    serial_loader u_rx (
        .clk(clk), .sck_pin(spi_sck), .sdi_pin(spi_sdi), .cs_n_pin(spi_cs_n),
        .a(a), .b(b), .op(op), .load_pulse(load_pulse)
    );

    alu_8bit u_alu (
        .a(a), .b(b), .op(op),
        .y(y), .zero(z), .carry(c), .overflow(v)
    );

    reg [7:0] result_q;
    reg       zero_q, carry_q, overflow_q;
    always @(posedge clk) begin
        result_q   <= y;
        zero_q     <= z;
        carry_q    <= c;
        overflow_q <= v;
    end
    assign result   = result_q;
    assign zero     = zero_q;
    assign carry    = carry_q;
```

```
    assign overflow = overflow_q;

endmodule
```

D.3 RP2040 firmware

Listing 4: RP2040 firmware; `/firmware/alu_test.py` in the project repository.

```
import shrike, time
from machine import Pin

shrike.flash("alu.bin")
time.sleep_ms(100)

SCK  = Pin(14, Pin.OUT, value=0)
SDI  = Pin(17, Pin.OUT, value=0)
CS_N = Pin(16, Pin.OUT, value=1)

OPS = {"ADD": 0b0000, "SLL": 0b0001, "SLT": 0b0010, "SLTU": 0b0011,
       "XOR": 0b0100, "SRL": 0b0101, "OR":  0b0110, "AND":  0b0111,
       "SUB": 0b1000, "SRA": 0b1101}

def send(op, a, b):
    """Send one 24-bit frame: {0000, op} A B, MSB first, SPI mode 0."""
    frame = ((op & 0xF) << 16) | ((a & 0xFF) << 8) | (b & 0xFF)
    CS_N(0); time.sleep_us(10)
    for i in range(23, -1, -1):
        SDI((frame >> i) & 1); time.sleep_us(5)
        SCK(1); time.sleep_us(5)
        SCK(0)
    time.sleep_us(10); CS_N(1)             # rising edge -> FPGA loads A, B, op

def s8(x):                                 # 8-bit two's complement -> int
    return x - 256 if x & 0x80 else x

def expected(name, a, b):
    sh = b & 7
    r = {"ADD": a + b, "SUB": a - b, "AND": a & b, "OR": a | b, "XOR": a ^ b,
         "SLL": a << sh, "SRL": a >> sh, "SRA": s8(a) >> sh,
         "SLT": int(s8(a) < s8(b)), "SLTU": int(a < b)}[name]
    return r & 0xFF

def demo(gap_ms=500):
    """One frame per operation; trigger the analyzer on CS_N rising."""
    tests = [("ADD", 0x7F, 0x01), ("SUB", 0x10, 0x20), ("AND", 0xF0, 0x3C),
             ("OR", 0xF0, 0x0F), ("XOR", 0xAA, 0xFF), ("SLL", 0x81, 0x03),
             ("SRL", 0x81, 0x03), ("SRA", 0x81, 0x03), ("SLT", 0xFF, 0x01),
             ("SLTU", 0xFF, 0x01)]
    for name, a, b in tests:
        send(OPS[name], a, b)
        print("%-4s A=%02X B=%02X -> expect %02X" % (name, a, b, expected(name, a, b))
    )
        time.sleep_ms(gap_ms)

A_PINS  = [5, 6, 7, 8, 9, 10, 11, 15]      # A0..A7  (A0 = LSB)
B_PINS  = [18, 19, 20, 21, 22, 23, 24, 25] # B0..B7
OP_PINS = [26, 27, 28, 29]                 # op0..op3
```

```
49
50  SWITCH_TO_GND = True
51
52  NAMES = {v: k for k, v in OPS.items()}
53
54  def live_mode():
55      pull = Pin.PULL_UP if SWITCH_TO_GND else None
56      pins = [Pin(p, Pin.IN, pull) for p in A_PINS + B_PINS + OP_PINS]
57
58      def read_all():
59          v = 0
60          for i, p in enumerate(pins):
61              bit = p.value()
62              if SWITCH_TO_GND:
63                  bit ^= 1                    # ON pulls the pin low -> count as 1
64              v |= bit << i
65          return v
66
67      last = None
68      print("Live mode: set A, B and op on the switches; the result follows.")
69      while True:
70          v = read_all()
71          time.sleep_ms(20)
72          if v == read_all() and v != last:  # same for 20 ms = settled, and changed
73              last = v
74              a, b, op = v & 0xFF, (v >> 8) & 0xFF, (v >> 16) & 0xF
75              send(op, a, b)
76              name = NAMES.get(op, "????")
77              exp = ("%02X" % expected(name, a, b)) if name in OPS else "00 (unused op)"
78              print("%-4s A=%02X B=%02X -> expect %s" % (name, a, b, exp))
79          time.sleep_ms(10)
80
81  #while True:
82  #    demo()
83  #demo()
84  live_mode()
```

E I/O Planner Assignment

Identical for both designs. Every output port is assigned to the OUT row of its GPIO and its `_oe` port to the OE row of the same GPIO.

Table 12: Complete I/O Planner assignment.

I/O Planner row	Port	Direction
OSC_CLK	`clk`	input
OSC_EN	`clk_en`	output
GPIO0_IN	`spi_cs_n`	input
GPIO7_IN	`spi_sdi`	input
GPIO18_IN	`spi_sck`	input
GPIO1_OUT / _OE	`zero` / `zero_oe`	output
GPIO2_OUT / _OE	`carry` / `carry_oe`	output
GPIO8–GPIO15 OUT / OE	`result[0..7]` / `result_oe[0..7]`	output
GPIO16_OUT / _OE	`overflow` / `overflow_oe`	output
GPIO3–GPIO6	– (used for bitstream loading)	–
GPIO17	– (unused)	–

F Wiring

Table 13: Board jumpers.

From	To	Signal
RP_IO14 (left header, pin 11)	FPGA_IO18 (left header, pin 15)	SCK
RP_IO17 (right header, pin 17)	FPGA_IO7 (left header, pin 19)	data
RP_IO16 (right header, pin 18)	FPGA_IO0 (left header, pin 16)	CS_N

Table 14: Analog Discovery 3 connections.

Analyzer	Board pin	Signal
GND	board GND	ground
DIO 0–7	FPGA_IO8–IO15 (PMOD)	result bits 0–7
DIO 8	FPGA_IO1	zero
DIO 9	FPGA_IO2	carry
DIO 11	FPGA_IO18	SCK
DIO 12	FPGA_IO7	data
DIO 13	FPGA_IO0	CS_N

Table 15: PMOD header layout (column 1 at the 3.3 V end).

	col 1	**col 2**	**col 3**	**col 4**	**col 5**	**col 6**
top	3.3 V	GND	IO8	IO10	IO12	IO14
bottom	3.3 V	GND	IO9	IO11	IO13	IO15

G WaveForms Configuration

Table 16: Logic analyzer settings.

Setting	**Value**
Bus	DIO 0–7 (DIO 0 = LSB), hexadecimal.
Signals	DIO 8 (zero), DIO 9 (carry).
SPI decoder	Select DIO 13 (active low), Clock DIO 11, Data DIO 12; sample on rising edge, 8 bits, MSB first, hexadecimal.
Trigger	Normal mode, falling edge of DIO 13.
Sample rate	1 MHz.
Time base	about 200 µs/div for a single frame.

Index

ALU, 1
Analog Discovery 3, 2

bit reversal, 5

carry chain, 5

firmware, 6
ForgeFPGA, 2

generate signal, 5
Go Configure, 2

instruction set architecture, 1

LUT, 8

opcode, 2
output enable, 4
overflow, 1, 3

pin assignment, 4
propagate signal, 5

readability indices, 13
reference model, 6
regular ALU, 4
resource utilisation, 7
resource-shared ALU, 4
RISC-V, 1
RP2040, 2

serial frame, 3
serial loader, 4
shifter, 5
Shrike-Lite, 2
slide switches, 5
SPI, 3
status flags, 3

test cases, 6
two's complement, 5

Verilog, 1

WaveForms, 2